

claude-windows / d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd\subagents\workflows\wf\_d2c10282-601\agent-a4b697c820517aa72.jsonl`
- SHA-256: `f0dacc545c6adb530f9fd367d84029b20928ff36939f8e9d5bcf63297baf9014`
- Source modified: `2026-07-04T11:54:03+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_d2c10282-601`
- session_id: `d4cf3fd6-b5bd-4ed1-9b57-88eea69333cd`

Transcript

1. user (2026-07-04T11:48:25.936Z)

You are drafting ONE section of the A13 "alpha launch review" for the KhelSutra badminton-highlight alpha
(repo Khelsutra/badminton-highlight-indexer, engine checkout at
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer; SPA/site repo at
C:/Users/avidu/Projects/khelsutra-guru/khelsutra).
Context: the A1 serving stack is LIVE on Cloud Run asia-southeast1 ? control plane service
'rally-control-plane' (rev 00007, edge-auth ON via Cloudflare Access at api.khelsutra.guru,
4-email
allowlist, maxScale=1, per-instance ephemeral SQLite state), L4 GPU Job 'rally-worker' (WASB-
only alpha:
indexing.skip_ai_handoff=true ? no Gemini in the serving config), storage gs://khelsutra-rally-
serving.
Deploy tooling: deploy/cloudrun/gen_service_yaml.py + README.md + Dockerfile; e2e evidence:
docs/COMPUTE_DECOUPLED_SERVING/runlogs/DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md (READ
IT).
Tracker: docs/ALPHA_LAUNCH_READINESS.md (\$7 rows; A10/A11 ship-gate + WASB-C3 provenance are
OWNER-GATED
and still open ? factor that into anything launch-facing).
READ-ONLY: do not modify any file, do not run git writes, do not touch GCP. Windows box,
PowerShell tool.
Ground EVERY claim in a file:line or issue number you actually opened ? no invented provenance.
If you
can't verify something, mark it [unverified]. Your output is a MARKDOWN SECTION consumed by an
orchestrator
assembling the review doc ? return it via the schema, self-contained, concise, evidence-cited.

SECTION: ****Observability & job-failure triage****. Trace the ACTUAL error paths in backend code
(backend/main.py, backend/orchestration.py, backend/api/job_worker.py, backend/api/uploads.py,
backend/compute/cloud_run.py, backend/infrastructure/database_jobs.py) and produce:
- a triage table: failure class -> what the CLIENT sees (status code + message shape) -> what
the OPERATOR
sees (log line pattern + which resource's logs) -> first debugging step. Cover at minimum:
input
validation failures, the #465 input-root jail rejection, staging failure, cloud dispatch
failure
(google-cloud-run missing/API error), L4 execution failure/timeout, bucket-poll timeout,
compile/ffmpeg
failure, signed-URL failure (now logged after #476 ? check the [gcs] warning + [highlights]
fallback),
upload 409/413/507, job stuck in 'queued'/'waiting', server restart mid-job
(recover_stale_jobs re-queue).
- correlation: how a client job_id maps to Cloud Run execution names/logs (dispatch token,
_dispatch done

markers) ? cite where the mapping is logged.
- GAPS list: what an operator CANNOT currently see (e.g. no request tracing #326, no per-video runlog
surface #328) with issue refs. Recommend the top 3 cheapest observability wins.

2. user (2026-07-04T11:48:37.230Z)

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend:

__pycache__
annotations
api
assets
audio
billing.py
compute
config
detectors
eval
features
flywheel.py
infrastructure
main.py
orchestration.py
pipeline
providers
reporting
results.py
server_runtime.py
sports
stitcher
storage
tools
ui
utils
worker

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api:

__init__.py
__pycache__
auth.py
job_worker.py
static
uploads.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute:

__init__.py
__pycache__
base.py
cloud_run.py

C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure:

__pycache__
database.py
database_jobs.py
database_media.py
database_user_models.py
execution_policy.py

3. user (2026-07-04T11:48:38.058Z)

No files found

4. user (2026-07-04T11:48:48.645Z)

```
DEPLOY_REPORT_cloud-serving_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md
DEPLOY_REPORT_cloud-serving_a1-e2e_2026-07-04.md
README.md
RUNLOG_truly-local_TEMPLATE.md
---
```

```
1649 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/main.py
867 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/orchestration.py
319 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/job_worker.py
212 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/api/uploads.py
311 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/compute/cloud_run.py
454 C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer/backend/infrastructure/database_jobs.py
3812 total
```

5. user (2026-07-04T11:48:52.953Z)

```
1      """Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3).
2
3      Extracted verbatim from backend.main to shrink the orchestrator; behavior is unchanged.
4      The worker DRAINS the jobs table via `claim_due_job` and honours WAIT_AND_RESUME: a job
5      that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
6      re-claimed when due ? with no central scheduler (the DB is the clock).
7
8      The monkeypatch contract these functions rely on (and that tests depend on):
9      every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
10     `_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is resolved THROUGH the
11     `backend.main` module object at call-time (`import backend.main as _main`), never as a
12     bare local. `backend.main` re-exports these names, so `monkeypatch.setattr(backend.main,
13     ...)`` still intercepts the inter-function calls (e.g. a parked job arms the `*patched*
14     `_arm_wake_timer`, so no real daemon timer leaks in tests). Importing `backend.main`
lazily
15     inside the functions also avoids the circular import at module load (main.py imports
this
16     module; this module must not import main at import time).
17     """
18
19     import json
20     import logging
21     import os
22     import threading
23     import traceback
24     from concurrent.futures import ThreadPoolExecutor
25     from typing import Any, Dict, Optional
26
27     logger = logging.getLogger(__name__)
28
29     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 by default: a single
30     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
31     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
32     # already parallelize their AI handoff internally via their own pools.
33     # EXCEPTION (#466): when the server CAN dispatch to Cloud Run (see
_cloud_dispatch_possible), a REMOTE
34     # job is only a bucket-poll (I/O-bound; GPU is on ephemeral L4s, WASB-only has no
Gemini), so the drain
35     # scales to deployment.max_concurrent_remote_jobs to overlap those. This does NOT
parallelize local
36     # work: CPU/GPU-heavy in-process detection + the compile stitch serialize via
37     # orchestration._LOCAL_COMPUTE_LANE regardless of the worker count, so the single-box
invariant holds
38     # and compile/local-override jobs stay serial. The count is fixed at first
```

```

_get_executor() creation.
39     #
40     # ? 02 RISK (CODE_CLEANUP_AUDIT §3c): a single worker means ONE stuck or hung job
41     # (e.g. hung Gemini generate, hung WSL GPU call, hung Cloud Run poll) blocks ALL
42     # subsequent jobs ? they stay 'queued' forever. Mitigations already in place:
43     #   - ExecutionPolicy op_timeout_sec kills hung generate/processing calls
44     #   - detector timeout_sec kills hung WSL/GPU calls
45     #   - Cloud Run poll has a max_wait_sec bound
46     #   - ffmpeg/ffprobe subprocesses use bounded media timeouts
47     # A future slice should add a per-job wall-clock timeout (the executor itself
48     # cannot enforce timeouts on the Thread).
49     _job_executor: Optional[ThreadPoolExecutor] = None
50     # Drain worker count, fixed at first _get_executor() creation. 1 (serial) by default;
configure_drain()
51     # raises it for cloud_run (#466). A module global (not a _get_executor arg) so the many
call sites +
52     # the tests that monkeypatch `_get_executor` as a no-arg lambda are unchanged.
53     _drain_max_workers: int = 1
54
55
56     def _cloud_dispatch_possible(config: Any = None) -> bool:
57         """True if this server CAN dispatch a job to Cloud Run ? either the profile sets
58         ``deployment.compute_target=cloud_run``, OR the per-request ``compute='cloud'``
override path is
59         wired (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT`` env + an output bucket). Mirrors
60         orchestration._cloud_available so drain concurrency tracks the ACTUAL remote-
dispatch capability,
61         not just the server default (so compute='cloud' overrides on a local-default server
parallelize too)."""
62         dep = getattr(config, "deployment", None)
63         if getattr(dep, "compute_target", "") == "cloud_run":
64             return True
65         if os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT"):
66             stor = getattr(config, "storage", None)
67             if (getattr(stor, "output_root", "") or "").strip():
68                 return True
69         return False
70
71
72     def _resolve_drain_workers(config: Any = None) -> int:
73         """Drain concurrency (#466): a server that CAN dispatch to Cloud Run (see
_get_cloud_dispatch_possible)
74         may run ``deployment.max_concurrent_remote_jobs`` drain workers so I/O-bound remote
dispatch/poll
75         OVERLAPS. This does NOT parallelize CPU/GPU-heavy LOCAL work ? in-process detection
and the compile
76         stitch serialize via orchestration._LOCAL_COMPUTE_LANE regardless of the worker
count ? so the
77         single-box GPU + rate-fragile Gemini invariant holds, and compile/local-override
jobs stay serial.
78         A server with no remote capability stays serial (1). Defaults to 1 when config is
absent
79         (tests / non-server) ? behaviour unchanged."""
80         if _cloud_dispatch_possible(config):
81             dep = getattr(config, "deployment", None)
82             return max(1, int(getattr(dep, "max_concurrent_remote_jobs", 1) or 1))
83         return 1
84
85
86     def configure_drain(config: Any = None) -> None:
87         """Set the drain worker count from ``config`` BEFORE the executor is created (#466).
Call once at
88         server startup. No-op effect once the executor already exists (the count is fixed at
creation)."""
89         global _drain_max_workers

```

```

90     _drain_max_workers = _resolve_drain_workers(config)
91
92
93     def _get_executor() -> ThreadPoolExecutor:
94         """Lazy module-global executor; tests monkeypatch this for inline execution. Worker
count is
95         ``_drain_max_workers`` (1 by default; raised for cloud_run via ``configure_drain``,
#466)."""
96         global _job_executor
97         if _job_executor is None:
98             _job_executor = ThreadPoolExecutor(
99                 max_workers=_drain_max_workers, thread_name_prefix="jobworker"
100             )
101         return _job_executor
102
103
104     class JobWaiting(Exception):
105         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
106         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
107         NOT a failure. The worker persists `next_poll_at` + `progress` (via
`set_job_waiting`),
108         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
date
109         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
contract.
110
111         The wiring is additive: the production segmenter path never raises this today (zero
112         behaviour change), so a job runs exactly as before. The hook is what a later slice
113         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
114         """
115
116         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
117             super().__init__(f"job parked for {delay_sec:.0f}s")
118             self.delay_sec = max(0.0, float(delay_sec))
119             self.progress = progress
120
121
122     def _job_to_request(job: Dict[str, Any]):
123         """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
124         re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
125         the original submit. The row stores exactly the ProcessRequest fields."""
126         import backend.main as _main
127
128         raw_params = job.get("params")
129         params = (
130             raw_params if isinstance(raw_params, dict) else json.loads(raw_params or "{}")
131         )
132         return _main.ProcessRequest(
133             video_path=job["video_path"],
134             player_pool=job.get("player_pool"),
135             max_frames=job.get("max_frames"),
136             segmenter=job.get("segmenter"),
137             # v8 compute-picker: the worker runs _execute_processing ?
138             _maybe_dispatch_remote, which
139             # honours this, so it MUST survive submit?reconstruct (unlike locale, which is
only read off
140             # the job row for analytics). NULL ? server default = pre-picker behaviour.
141             compute=job.get("compute"),
142             force=bool(params.get("force")),
143         )
144
145     def _run_claimed_job(job: Dict[str, Any], config: Any, db: Any) -> None:
146         """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
its

```

```

147     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
148
149     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
means
150     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
151     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
the
152     job self-scheduled and will be re-claimed when `next_poll_at` is due.
153     """
154     import backend.main as _main
155
156     job_id = job["id"]
157     # Dispatch by kind: 'compile' (#329, the async reel stitch) vs 'process'
(NULL/default ? the
158     # original /api/process pipeline). Both record their terminal state the same way;
only process
159     # jobs feed the M8 cost ledger + M11 flywheel (a compile has no detection
usage/segments).
160     is_compile = job.get("kind") == "compile"
161     try:
162         if is_compile:
163             result = _main._execute_compile(
164                 config,
165                 db,
166                 job,
167                 progress=lambda stage: db.set_job_progress(job_id, stage),
168             )
169         else:
170             req = _main._job_to_request(job)
171             result = _main._execute_processing(
172                 config,
173                 db,
174                 req,
175                 progress=lambda stage: db.set_job_progress(job_id, stage),
176             )
177             if result.get("video_id"):
178                 db.set_job_video_id(job_id, result["video_id"])
179             db.mark_job_done(job_id, result)
180             if not is_compile:
181                 _maybe_record_job_cost(
182                     job, result, config, db
183                 ) # M8 cost ledger (default-OFF; process jobs only)
184                 _maybe_run_flywheel(
185                     job, result, config
186                 ) # M11 data-flywheel (process jobs only)
187     except _main.JobWaiting as w:
188         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
189         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
190         db.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
191     except Exception as e:
192         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
193         db.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
194
195
196     def _maybe_record_job_cost(
197         job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
198     ) -> None:
199         """M8 cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8). DEFAULT-OFF: a no-op unless
200         ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
201         (= {gpu_seconds, wall_seconds, egress_gb, gemini_usd, gpu_type}) × the versioned
pricebook, and
202         persist it.
203
204         ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet,
so even with

```

```

205         ``billing.enabled=True`` AND a real calibrated pricebook, every recorded cost is
**0** until a
206     FOLLOW-UP wires the worker to emit usage (gpu/wall/egress/gemini). M8 lands the
ledger + pricebook
207     + the recording seam; the usage-emission step is separate (the metering hooks on the
real run).
208     With the placeholder pricebook the cost is 0 regardless.
209
210     Best-effort ? never fails a completed job because cost bookkeeping hiccuped. ``job``
is the
211     claimed row (carries ``owner_id``); ``result`` is the pipeline output.""
212
213     billing_cfg = getattr(config, "billing", None)
214     if not billing_cfg or not getattr(billing_cfg, "enabled", False):
215         return
216     try:
217         usage = (result or {}).get("usage") or {}
218         db.record_job_cost(
219             job["id"],
220             usage=usage,
221             pricebook_version=billing_cfg.pricebook_version,
222             gpu_type=usage.get("gpu_type"),
223             compute_backend=(result or {}).get("compute_backend"),
224             owner_id=job.get("owner_id"),
225             margin=billing_cfg.margin,
226         )
227         logger.info(
228             "Recorded cost for job %s (pricebook=%s).",
229             job["id"],
230             billing_cfg.pricebook_version,
231         )
232     except Exception as e: # never wedge a completed job on bookkeeping
233         logger.warning(
234             "Cost recording for job %s failed (non-fatal): %s", job.get("id"), e
235         )
236
237
238     def _maybe_run_flywheel(
239         job: Dict[str, Any], result: Dict[str, Any], config: Any
240     ) -> None:
241         """M11 data-flywheel (COMPUTE_DECOUPLED_SERVING M11, D12). DEFAULT-OFF + INERT: a
no-op unless
242         ``config.flywheel.enabled`` AND attested consent ? and consent is a **faked hard-
deny** today
243         (``flywheel.consent_attested`` returns False for every job until counsel + the
consent schema
244         land), so this captures **nothing** in production. When activated it captures the
consented job's
245         artifacts into the per-video workspace (? later labeling ? the golden training set).
246
247         Best-effort ? never fails a completed job because the flywheel hiccuped. ``job`` is
the claimed
248         row (carries ``owner_id``); ``result`` is the pipeline output.""
249
250         flywheel_cfg = getattr(config, "flywheel", None)
251         if not flywheel_cfg or not getattr(flywheel_cfg, "enabled", False):
252             return
253         try:
254             from backend import flywheel
255
256             flywheel.run_for_job(job, result, config)
257         except Exception as e: # never wedge a completed job on the flywheel
258             logger.warning("Flywheel for job %s failed (non-fatal): %s", job.get("id"), e)
259
260

```

```

261     def _drain_due_jobs(config: Any, db: Any) -> int:
262         """One worker tick: claim and run every currently-runnable job (queued, or waiting
whose
263         `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
264
265         Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
picked
266         up by a FUTURE drain once due; we schedule that follow-up drain via
`_schedule_drain` so
267         a single-worker box keeps making progress with no central timer (the DB is the
clock).
268         Returns the number of jobs that reached a terminal/parked state this tick.
269         """
270         import backend.main as _main
271
272         ran = 0
273         while True:
274             job = db.claim_due_job()
275             if job is None:
276                 break
277             ran += 1
278             _main._run_claimed_job(job, config, db)
279         # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
new
280         # submit. On the single-box executor this is a delayed re-drain; multi-worker just
relies
281         # on the next submit/drain. Best-effort ? never raises into the worker.
282         _main._schedule_next_drain_if_waiting(config, db)
283         return ran
284
285
286     def _schedule_next_drain_if_waiting(config: Any, db: Any) -> None:
287         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
288         self-scheduled job resumes with no further client submit. The drain itself re-checks
289         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
290         import backend.main as _main
291
292         try:
293             delay = db.seconds_until_next_due()
294         except Exception as e: # pragma: no cover - defensive; never wedge the worker
295             logger.error(f"Could not compute next-due delay: {e}")
296             return
297         if delay is None:
298             return # nothing waiting
299         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
300         _main._arm_wake_timer(max(0.0, min(delay, _main._MAX_DRAIN_WAKE_SEC)), config, db)
301
302
303     def _arm_wake_timer(delay_sec: float, config: Any, db: Any) -> None:
304         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
305         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
306         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
307         import backend.main as _main
308
309         timer = threading.Timer(
310             delay_sec,
311             lambda: _main._get_executor().submit(_main._drain_due_jobs, config, db),
312         )
313         timer.daemon = True
314         timer.start()

```



```

315
316
317 #: A parked job further out than this is re-checked by a capped wake rather than one
long
318 #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
319 _MAX_DRAIN_WAKE_SEC = 60.0
320

```

6. user (2026-07-04T11:48:53.425Z)

```

1      """`CloudRunCompute` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2      for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4      Flow (the cloud-serving compute path):
5      1. **Dispatch** a Cloud Run Job execution whose container runs `python -m
backend.worker infer
6          --video-uri <gs> --out-uri <gs> --done-uri <gs> ?` (the SAME worker, on the L4
GPU).
7      2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8          completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9          it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10         cold-start is absorbed by the async poll. On a poll TIMEOUT (``max_wait_sec`` can
undercut the
11         job's own ``--task-timeout`` budget) the shim does ONE final marker check ? a slow-
but-healthy
12         worker that finished just past the deadline is rescued, not wasted ? then best-
effort CANCELS
13         the still-running execution so the L4 doesn't keep billing until ``--task-
timeout``.
14     3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
15         ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
16
17     The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
18     so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
19
20     The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
21     with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
22     owner-verified on the M7 real run.
23     """
24
25     from __future__ import annotations
26
27     import json
28     import logging
29     import os
30     import tempfile
31     import uuid
32     from abc import ABC, abstractmethod
33     from typing import Any, List, Optional
34
35     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
36     from backend.infrastructure.execution_policy import (
37         DEFAULT_POLICY,
38         ExecutionPolicy,
39         PolicyTimeout,
40         poll_until,
41     )

```

```

42     from backend.storage import fetch, get_storage, parse_uri
43
44     LOG = logging.getLogger("compute.cloud_run")
45
46     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
47     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
48     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
49         "deployment.compute_target=local_gpu",
50         "indexing.detector_impl=native",
51     )
52
53
54     class CloudRunJobClient(ABC):
55         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
56
57         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
58         overrides the job's container ``args`` for this execution and starts it."""
59
60         @abstractmethod
61         def run_job(
62             self,
63             job_name: str,
64             argv: List[str],
65             *,
66             region: str,
67             project: Optional[str] = None,
68         ) -> str:
69             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
70             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
71
72         @abstractmethod
73         def cancel_execution(self, execution: str) -> None:
74             """Cancel a running execution by the name ``run_job`` returned. Called when the
bucket-poll
75             times out so the abandoned execution doesn't keep billing the GPU until
``--task-timeout``.
76             May raise ? the caller treats every failure as best-effort (the original poll
timeout is
77             NEVER masked by a cancel error)."""
78
79
80     class GoogleCloudRunJobClient(CloudRunJobClient):
81         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
82         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
83
84         def run_job(
85             self,
86             job_name: str,
87             argv: List[str],
88             *,
89             region: str,
90             project: Optional[str] = None,
91         ) -> str:
92             # A None/empty project would build a bogus "projects/None/locations/..." path
(job_path
93             # requires a real str). Fail loudly BEFORE the SDK import so the misconfig is
exercisable

```

```

94         # in CI (where google-cloud-run is absent) and so mypy narrows project to str
for job_path.
95         if not project:
96             raise RuntimeError(
97                 "GOOGLE_CLOUD_PROJECT is unset ? the real Cloud Run dispatch needs a GCP
project id "
98                 "to resolve the job path; set it on the control plane (see
get_compute_backend)."
99             )
100         try:
101             from google.cloud import run_v2 # type: ignore[attr-defined]
102         except Exception as exc: # pragma: no cover - real SDK not present in CI
103             raise RuntimeError(
104                 "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
105                 "control plane (it is intentionally NOT a CI/test dependency)."
106             ) from exc
107         client = (
108             run_v2.JobsClient()
109         ) # pragma: no cover - exercised only on the real M7 run
110         name = client.job_path(project, region, job_name)
111         overrides = run_v2.RunJobRequest.Overrides(
112             container_overrides=[
113                 run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)
114             ]
115         )
116         op = client.run_job(
117             request=run_v2.RunJobRequest(name=name, overrides=overrides)
118         )
119         return (
120             getattr(op, "metadata", None)
121             and getattr(op.metadata, "name", "")
122             or "execution"
123         )
124
125     def cancel_execution(self, execution: str) -> None:
126         # run_job falls back to the literal string "execution" when the operation
metadata carries
127         # no name ? that is NOT a resolvable resource name. Fail loudly BEFORE the SDK
import
128         # (mirrors run_job's project guard, #342) so the misuse is exercisable in CI
(where
129         # google-cloud-run is absent).
130         if "/executions/" not in (execution or ""):
131             raise RuntimeError(
132                 f"not a fully-qualified Cloud Run execution name: {execution!r} ?
expected "
133                 "projects/<p>/locations/<l>/jobs/<j>/executions/<e> (run_job could not
resolve "
134                 "one from the operation metadata, so there is nothing to cancel by
name)."
135             )
136         try:
137             from google.cloud import run_v2 # type: ignore[attr-defined]
138         except Exception as exc: # pragma: no cover - real SDK not present in CI
139             raise RuntimeError(
140                 "google-cloud-run is required to cancel a Cloud Run execution; install
it on the "
141                 "control plane (it is intentionally NOT a CI/test dependency)."
142             ) from exc
143         client = (
144             run_v2.ExecutionsClient()
145         ) # pragma: no cover - exercised only on a real run
146         client.cancel_execution(request=run_v2.CancelExecutionRequest(name=execution))
147

```

```

148
149 class CloudRunCompute(ComputeBackend):
150     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
151
152     def __init__(
153         self,
154         *,
155         run_client: CloudRunJobClient,
156         job_name: str,
157         region: str,
158         project: Optional[str] = None,
159         storage_config: Optional[dict] = None,
160         policy: ExecutionPolicy = DEFAULT_POLICY,
161         token: Optional[str] = None,
162         container_overrides: Optional[tuple[str, ...]] = None,
163     ) -> None:
164         self._client = run_client
165         self._job_name = job_name
166         self._region = region
167         self._project = project
168         self._storage_config = storage_config or {}
169         self._policy = policy
170         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
171         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
172         self._token = token
173         self._container_overrides = (
174             _DEFAULT_CONTAINER_OVERRIDES
175             if container_overrides is None
176             else tuple(container_overrides)
177         )
178
179     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
180         out_root = spec.out_uri.rstrip("/")
181         token = self._token or uuid.uuid4().hex
182         done_uri = f"{out_root}/_dispatch/{token}.done"
183         argv: List[str] = [
184             "infer",
185             "--video-uri",
186             spec.video_uri,
187             "--out-uri",
188             spec.out_uri,
189             "--done-uri",
190             done_uri,
191         ]
192         if spec.segmenter:
193             argv += ["--segmenter", spec.segmenter]
194         for kv in (*spec.config_overrides, *self._container_overrides):
195             argv += ["--set", kv]
196
197         execution = self._client.run_job(
198             self._job_name, argv, region=self._region, project=self._project
199         )
200         LOG.info(
201             "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
202             self._job_name,
203             execution,
204             done_uri,
205         )
206
207         try:
208             marker = poll_until(
209                 lambda: self._read_marker(done_uri),
210                 self._policy,

```

```

211         label="cloud-run-infer",
212         logger=LOG,
213     )
214 except PolicyTimeout as timeout_exc:
215     marker = self._handle_poll_timeout(done_uri, str(execution), timeout_exc)
216     video_id = str(marker.get("video_id", ""))
217     # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
218     # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
219     rc = int(marker.get("returncode", 1))
220     if not video_id:
221         LOG.warning(
222             "cloud-run: job=%s completion marker present but empty/unreadable (no "
223             "video_id) ? treating as failure",
224             self._job_name,
225         )
226     rc = rc or 1
227     outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
228     report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
229     LOG.info(
230         "cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id, rc
231     )
232     return ComputeResult(
233         returncode=rc,
234         outputs_uri=outputs_uri,
235         video_id=video_id,
236         report=report,
237         execution=str(execution or ""),
238     )
239
240 def _handle_poll_timeout(
241     self, done_uri: str, execution: str, timeout_exc: PolicyTimeout
242 ) -> dict:
243     """Poll deadline hit ? rescue a just-finished result, else cancel + re-raise.
244
245     The poll budget (`max_wait_sec`, default 1800s) can undercut the deployed
job's own
246     ``--task-timeout`` (3600s), so a slow-but-healthy worker may finish just past
the deadline:
247     ONE final marker check rescues that completed GPU run instead of wasting it. If
the marker
248     is genuinely absent, the execution is still running ? best-effort cancel it
(otherwise the
249     L4 keeps billing until ``--task-timeout``) and re-raise a ``PolicyTimeout``
carrying the
250     execution name so the operator can verify the state in the GCP console. A cancel
failure
251     NEVER masks the original timeout."""
252     try:
253         late = self._read_marker(done_uri)
254     except Exception: # noqa: BLE001 ? the final check is best-effort too: absence
? failure
255         LOG.warning(
256             "cloud-run: final post-timeout marker check failed ? treating as
absent",
257             exc_info=True,
258         )
259         late = None
260     if late is not None:
261         LOG.warning(
262             "cloud-run: job=%s exec=%s done-marker found on the final post-timeout
check "
263             "? rescuing the completed result",
264             self._job_name,

```

```

265         execution,
266     )
267     return late
268 try:
269     self._client.cancel_execution(execution)
270 except Exception as cancel_exc: # noqa: BLE001 ? best-effort: never mask the
timeout
271     cancel_note = f"best-effort cancel FAILED ({cancel_exc})"
272     LOG.warning(
273         "cloud-run: best-effort cancel of execution=%s failed: %s",
274         execution,
275         cancel_exc,
276         exc_info=True,
277     )
278 else:
279     cancel_note = "cancel requested"
280     LOG.info("cloud-run: requested cancel of execution=%s", execution)
281     raise PolicyTimeout(
282         f"{timeout_exc} [cloud-run job={self._job_name} "
283         f"execution={execution or '<unknown>'}; {cancel_note} ? verify the execution
"
284         "state in the GCP console]"
285     ) from timeout_exc
286
287 # --- storage-backed completion signal (bucket-poll) ---
288 def _read_marker(self, done_uri: str) -> Optional[dict]:
289     """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
290     ref = parse_uri(done_uri)
291     if not get_storage(ref.backend, config=self._storage_config).exists(ref):
292         return None
293     return (
294         self._read_json(done_uri) or {}
295     ) # empty dict still terminates the poll (present == done)
296
297 def _read_json(self, uri: str) -> dict:
298     # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
299     # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
300     # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
301     try:
302         ref = parse_uri(uri)
303         with tempfile.TemporaryDirectory(prefix="cr-read-") as td:
304             dst = os.path.join(td, ref.name or "obj.json")
305             local = fetch(uri, dst, config=self._storage_config)
306             with open(local, "r", encoding="utf-8") as f:
307                 data: Any = json.load(f)
308             return data if isinstance(data, dict) else {}
309     except Exception:
310         LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
311         return {}
312

```

7. user (2026-07-04T11:49:02.718Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.

```

```

9
10 The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11 ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12 reads the live value the server set at startup (and so tests that monkeypatch
13 ``backend.main.global_config`` flow through unchanged).
14 """
15
16 import os
17 import threading
18 import unicodedata
19 import uuid
20 from typing import Any, Dict, Optional
21
22 from fastapi import APIRouter, HTTPException, Request
23 from pydantic import BaseModel
24
25 from backend.utils.app_home import (
26     resolve_output_dir, # P0a: app-home-aware path resolution
27 )
28 from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
29 from backend.utils.hashing import compute_video_id # canonical file-identity hashing
30 from backend.utils.paths import safe_output_filename
31
32 router = APIRouter()
33
34 _uploads: Dict[str, Dict[str, Any]] = {}
35 _uploads_lock = threading.Lock()
36
37
38 class UploadInitRequest(BaseModel):
39     filename: str
40     total_size_bytes: Optional[int] = None
41
42
43 class UploadCompleteRequest(BaseModel):
44     total_parts: int
45
46
47 def _upload_cfg():
48     # Resolved lazily (import inside the function) to read the LIVE startup config that
49     # main() sets on the module global, and to avoid a circular import at module load
50     # (main.py imports this router; this module must not import main at import time).
51     import backend.main as _main
52
53     sec = getattr(_main.global_config, "security", None)
54     upload_dir = resolve_output_dir(
55         getattr(sec, "upload_dir", None) or "output/uploads"
56     ) # P0a: app-home-rooted (CWD-identical when RALLY_APP_HOME unset)
57     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
58     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
59     exts = [
60         e.lower().lstrip(".")
61         for e in (getattr(sec, "allowed_upload_extensions", None) or ["mp4", "mov"])
62     ]
63     return upload_dir, max_bytes, part_bytes, exts
64
65
66 def _abort_upload(upload_id: str) -> None:
67     with _uploads_lock:
68         st = _uploads.pop(upload_id, None)
69         if st:
70             try:
71                 os.remove(st["tmp_path"])
72             except OSError:
73                 pass

```

```

74
75
76     def _normalize_upload_filename(filename: str) -> str:
77         """Normalize browser-provided file names before extension validation.
78
79         Some mobile/file-provider pickers can hand us visually normal names with
compatibility
80         characters (for example a fullwidth dot) or trailing whitespace/dots. Keep the
existing
81         traversal/null-byte checks in ``safe_output_filename`` as the authority after this
cleanup.
82         """
83         return unicodedata.normalize("NFKC", filename).strip().rstrip(". ")
84
85
86     @router.post("/api/upload/init")
87     def upload_init(req: UploadInitRequest):
88         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
89         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
90         try:
91             name = safe_output_filename(_normalize_upload_filename(req.filename))
92         except ValueError as e:
93             raise HTTPException(status_code=400, detail=str(e)) from e
94         ext = os.path.splitext(name)[1].lower().lstrip(".")
95         if ext not in exts:
96             raise HTTPException(
97                 status_code=400,
98                 detail=f"Extension '{ext}' not allowed. Allowed: {sorted(exts)}",
99             )
100         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
101             raise HTTPException(
102                 status_code=413,
103                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB upload limit.",
104             )
105         os.makedirs(upload_dir, exist_ok=True)
106         # P2 (D11): fail fast with 507 if the box can't hold the declared upload. Best-
effort ? an
107         # undeclared size (total_size_bytes=None) or an unreadable disk skips the guard
(never blocks a
108         # legit upload); the per-part total cap in upload_part is the hard backstop either
way.
109         require_free_bytes(upload_dir, req.total_size_bytes, stage="upload")
110         upload_id = uuid.uuid4().hex
111         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
112         open(tmp_path, "wb").close()
113         with _uploads_lock:
114             _uploads[upload_id] = {
115                 "filename": name,
116                 "tmp_path": tmp_path,
117                 "parts": 0,
118                 "bytes": 0,
119             }
120         return {
121             "upload_id": upload_id,
122             "max_part_mb": part_bytes // (1024 * 1024),
123             "next_part": f"/api/upload/{upload_id}/part/0",
124         }
125
126
127     @router.put("/api/upload/{upload_id}/part/{index}")
128     async def upload_part(upload_id: str, index: int, request: Request):
129         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
130         _, max_bytes, part_bytes, _ = _upload_cfg()
131         with _uploads_lock:
132             st = _uploads.get(upload_id)

```



```

133         if st is None:
134             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
135         if index != st["parts"]:
136             raise HTTPException(
137                 status_code=409,
138                 detail=f"Out-of-order part: expected {st['parts']}, got {index} (parts are
sequential)",
139             )
140
141         received = 0
142         overflow = None
143         with open(st["tmp_path"], "ab") as fh:
144             async for chunk in request.stream():
145                 received += len(chunk)
146                 if received > part_bytes:
147                     overflow = (
148                         f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part limit."
149                     )
150                     break
151                 if st["bytes"] + received > max_bytes:
152                     overflow = (
153                         f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total limit."
154                     )
155                     break
156                 fh.write(chunk)
157         if overflow:
158             _abort_upload(upload_id)
159             raise HTTPException(
160                 status_code=413, detail=overflow + " Upload aborted ? re-init to retry."
161             )
162
163         with _uploads_lock:
164             st["parts"] += 1
165             st["bytes"] += received
166         return {
167             "upload_id": upload_id,
168             "received_parts": st["parts"],
169             "received_bytes": st["bytes"],
170         }
171
172
173     @router.post("/api/upload/{upload_id}/complete")
174     def upload_complete(upload_id: str, req: UploadCompleteRequest):
175         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
176         upload_dir, _, _, _ = _upload_cfg()
177         with _uploads_lock:
178             st = _uploads.get(upload_id)
179         if st is None:
180             raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
181         if st["parts"] == 0 or st["parts"] != req.total_parts:
182             raise HTTPException(
183                 status_code=409,
184                 detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
185             )
186         final_path = os.path.join(upload_dir, st["filename"])
187         if os.path.exists(
188             final_path
189         ): # never overwrite an earlier upload with the same name
190             stem, ext = os.path.splitext(st["filename"])
191             final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
192         os.replace(st["tmp_path"], final_path)
193         with _uploads_lock:
194             _uploads.pop(upload_id, None)
195         video_id = compute_video_id(final_path)

```

```

196         return {
197             "path": os.path.abspath(final_path),
198             "video_id": video_id,
199             "size_bytes": os.path.getsize(final_path),
200             "next": "POST /api/process with this path",
201         }
202
203
204     @router.delete("/api/upload/{upload_id}")
205     def upload_abort(upload_id: str):
206         """Abort an in-flight upload and delete its partial file."""
207         with _uploads_lock:
208             known = upload_id in _uploads
209             if not known:
210                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
211             _abort_upload(upload_id)
212             return {"status": "aborted", "upload_id": upload_id}
213

```

8. user (2026-07-04T11:49:08.024Z)

```

1     """Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of
#390).
2
3     The full hash -> validate -> segment -> report pipeline (``_execute_processing``), the
4     Cloud Run dispatch + ingest path (``_maybe_dispatch_remote`` /
``_ingest_remote_segments``),
5     and the shared compile resolution + worker handler (``_resolve_compile`` /
``_execute_compile``).
6     Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is
unchanged.
7
8     SEAM DISCIPLINE (same convention as ``backend.api.job_worker``): anything that must
remain
9     monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional
10     ``db_instance``/``global_config`` globals and the ``ProcessRequest`` model) is resolved
through
11     the ``backend.main`` module object at call time (``import backend.main as _main``),
never as a
12     bare local. ``backend.main`` re-exports every name below, so
``monkeypatch.setattr(backend.main,
13     ...)` and ``import backend.main as m; m._execute_processing(...)`` keep working
unchanged.
14     This module must NOT import ``backend.main`` at module level (main imports us for the
re-export).
15     Routed seams (rebind-patched on ``backend.main`` by tests): ``global_config``,
``db_instance``,
16     ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
17     Object-attribute patches (``m.storage.fetch``, ``m.segmenter_registry.get``, ...) hit
the shared
18     objects and need no routing.
19     """
20
21     import csv
22     import json
23     import logging
24     import math
25     import os
26     import tempfile
27     import threading
28     from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
29     from urllib.parse import quote
30
31     from backend import storage
32     from backend.config import AppConfig, StitchingConfig

```

```

33     from backend.pipeline.segmenters.base import segmenter_registry
34     from backend.pipeline.stitcher.watermark_i18n import localize_watermark
35     from backend.pipeline.validators.base import registry
36     from backend.results import Failure
37     from backend.utils.paths import output_base, safe_output_filename
38
39     if (
40         TYPE_CHECKING
41     ): # runtime import would be circular (main imports us for the re-export)
42         from backend.main import ProcessRequest # noqa: F401
43
44     logger = logging.getLogger(__name__)
45
46     # #466: serialize CPU/GPU-heavy LOCAL work ? in-process segmentation and the ffmpeg
compile stitch ?
47     # so that raising the drain's worker count (to overlap I/O-bound REMOTE Cloud Run
dispatch/poll) never
48     # accidentally parallelizes local work. A single self-hosted box serializes GPU work and
Gemini is
49     # rate-fragile; a remote dispatch is a bucket-poll and never holds this lane, so N
remote jobs overlap
50     # while local/compile jobs stay 1-at-a-time. It's a plain mutex (no nesting) ? no
deadlock; when the
51     # drain is serial (max_workers=1) it's an uncontended no-op = today's behaviour.
52     _LOCAL_COMPUTE_LANE = threading.BoundedSemaphore(1)
53
54
55     def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
56         """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
57
58         On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
59         fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
and
60         keep the prior good results intact ? a failed/empty re-run never destroys prior
data.
61         """
62         if segments:
63             db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
64         else:
65             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
66
67
68     def _validation_failure_message(failures: List[Any]) -> str:
69         details = []
70         for failure in failures:
71             name = getattr(failure, "validator_name", None) or "validation"
72             message = (getattr(failure, "message", None) or "").strip()
73             details.append(f"{name}: {message}" if message else str(name))
74         if not details:
75             return "Video failed validation checks."
76         return "Video failed validation: " + "; ".join(details)
77
78
79     def _report_config_for_input(config: Any, raw_input: str) -> Any:
80         """Return report config that keeps remote-input reports out of disposable
scratch."""
81         storage_config = (
82             config.get("storage")
83             if isinstance(config, dict)
84             else getattr(config, "storage", None)
85         )
86         try:
87             if storage.input_is_local(raw_input, storage_config):
88                 return config
89         except ValueError:

```

```

90         return config
91     try:
92         cfg = (
93             config.model_dump(mode="json")
94             if hasattr(config, "model_dump")
95             else dict(config or {})
96         )
97     except Exception:
98         cfg = {}
99     report = dict(cfg.get("report") or {})
100     if report.get("output_dir"):
101         return config
102     report["output_dir"] = os.path.join(output_base(config), "reports")
103     cfg["report"] = report
104     return cfg
105
106
107 def _ingest_remote_segments(
108     db, video_id: str, outputs_uri: str, storage_config: dict, *, max_rows: int = 20000
109 ) -> int:
110     """Read a cloud job's ``intervals.csv`` (start,end,shot_count,ending_reason) from
111     the bucket and
112     persist each rally via the SAME ``db.add_play_segment`` the in-process segmenters
113     call ? so
114     ``/api/query`` + ``/api/compile`` read the rows with zero knowledge of where they
115     were produced.
116     Returns the count ingested. Mirrors ``cloud_run._read_json``'s fetch-to-a-real-dst
117     pattern (cloud
118     backends REQUIRE a dst). Hardened against a hostile/corrupt worker output: a
119     malformed row is
120     skipped, non-finite / overflowing values are rejected, the loop is bounded, and the
121     temp dir is
122     cleaned. May raise (missing/unreadable CSV, fetch error) ? the caller treats that as
123     a job failure
124     and prunes any partial rows (destroy-AFTER-confirm)."""
125     uri = f"{outputs_uri.rstrip('/')}/intervals.csv"
126     ref = storage.parse_uri(uri)
127     n = 0
128     with tempfile.TemporaryDirectory(prefix="cr-ingest-") as td:
129         local = storage.fetch(
130             uri, os.path.join(td, ref.name or "intervals.csv"), config=storage_config
131         )
132         with open(local, newline="", encoding="utf-8") as f:
133             for row in csv.DictReader(f):
134                 if n >= max_rows:
135                     logger.warning(
136                         "[API] cloud ingest hit the %d-row cap (%s) ? ignoring the
137                         rest.",
138                         max_rows,
139                         uri,
140                     )
141                     break
142                 try:
143                     start, end = float(row["start"]), float(row["end"])
144                 except (KeyError, TypeError, ValueError):
145                     continue
146                 if not (math.isfinite(start) and math.isfinite(end)):
147                     continue # reject inf/nan times rather than persist a junk rally
148                 meta: Dict[str, Any] = {
149                     "source": "cloud_run",
150                     "ending_reason": (row.get("ending_reason") or "").strip(),
151                 }
152                 sc = (row.get("shot_count") or "").strip()
153                 if sc:
154                     try:

```

```

147         meta["shot_count"] = int(
148             float(sc)
149         ) # OverflowError on 'inf'/'1e400'
150     except (ValueError, OverflowError):
151         meta["shot_count"] = sc
152     db.add_play_segment(video_id, start, end, metadata=meta)
153     n += 1
154     return n
155
156
157 def _cloud_available(cfg) -> bool:
158     """Whether THIS server can satisfy a per-request ``compute="cloud"`` dispatch (item
159 3).
160     True when the control plane is wired for Cloud Run: either the configured default
161     target is already ``cloud_run`` (the operator chose cloud serving), or the Cloud Run env
162     coordinates (``CLOUD_RUN_JOB`` + ``GOOGLE_CLOUD_PROJECT``) and an output bucket
163     (``storage.output_root``)
164     are present so a forced per-request override can actually dispatch + collect a
165     result.
166     Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
167     would work."""
168     deployment = getattr(cfg, "deployment", None)
169     if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
170         return True
171     has_job = bool(
172         os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
173     )
174     out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
175     return bool(has_job and out_root)
176
177 def _maybe_dispatch_remote(
178     cfg,
179     db,
180     req: "ProcessRequest",
181     video_id: str,
182     seg_name: str,
183     val_results,
184     play_wm: int,
185     cand_wm: int,
186     notify,
187 ) -> Optional[Tuple[Dict[str, Any], bool]]:
188     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
189     ``deployment.compute_target
190     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
191     intervals into
192     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
193     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
194     the cloud job
195     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
196     observability report
197     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
198     process segmenter
199     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
200     non-``cloud_run`` target, so
201     the in-process path stays byte-identical)."""
202     from backend.compute import ComputeJobSpec, get_compute_backend
203
204     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
205         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows

```

```

(id > play_wm);
200         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
201         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
202         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
only when the
203         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
rc / ingest
204         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
records the
205         # intended backend, `requested` the picker choice, `dispatched` whether the
remote job ran.
206         return (
207             {
208                 "video_id": video_id,
209                 "status": "failed",
210                 "message": msg,
211                 "results": [r.model_dump() for r in val_results],
212                 "play_segments": [],
213                 "compute": {
214                     "path": "cloud_run" if ran else "not_run",
215                     "requested": req.compute,
216                     "target": "cloud_run",
217                     "dispatched": ran,
218                 },
219             },
220             ran,
221         )
222
223         # SPA compute-picker (item 3): a per-request choice overrides the server's
configured default.
224         # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
(guarded by
225         # _cloud_available so we fail clearly rather than silently running local); None /
unknown ? default.
226         choice = (req.compute or "").strip().lower()
227         if choice and choice not in ("local", "cloud"):
228             logger.warning(
229                 "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server
default.",
230                 req.compute,
231             )
232         choice = ""
233         if choice == "local":
234             return None # explicit "run on my machine" ? in-process regardless of the
server's target
235         if choice == "cloud":
236             if not _cloud_available(cfg):
237                 return _failed(
238                     "cloud-serving was requested but this server isn't configured for it "
239                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
240                     False,
241                 )
242             compute = get_compute_backend(cfg, target_override="cloud_run")
243         else:
244             compute = get_compute_backend(
245                 cfg
246             ) # server default (default-OFF unless cloud_run)
247         if compute is None:
248             return None # default-OFF ? run the in-process segmenter below (unchanged)
249
250         storage_cfg = getattr(cfg, "storage", None)
251
252         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.

```

```

253     try:
254         input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
255     except ValueError as e:
256         return _failed(
257             f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
258         )
259     if input_ref.backend == "local":
260         # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
261         # can't read it. Stage it UP to the input bucket first, then dispatch from there
262         # possible when a CLOUD input_root is configured; without one we genuinely can't
263         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
264         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
265         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
266         # default, while a bare/local-looking root can't stage no matter what
input_backend says
267         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
268         # gives us nothing to build the staged URI under).
269         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
270     try:
271         root_backend = (
272             storage.parse_uri(input_root).backend if input_root else "local"
273         )
274     except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
275         root_backend = "local"
276     if root_backend == "local":
277         return _failed(
278             "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
279             "stage local uploads into; a path local to this machine can't be read by
the Cloud "
280             "Run job.",
281             False,
282         )
283     local_src = input_ref.key # resolved local path of the upload
284     staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
285     try:
286         notify("staging upload to bucket")
287         # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
288         # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
289         storage.put(
290             local_src,
291             staged_uri,
292             config=(
293                 storage_cfg.model_dump() if storage_cfg is not None else None
294             ),
295         )
296     except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500
297         logger.error(
298             "[API] cloud-serving: failed to stage %s -> %s: %s",
299             local_src,
300             staged_uri,
301             e,
302             exc_info=True,

```

```

303         )
304         return _failed(
305             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
306             False,
307         )
308         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
309         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
310         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
311         req.video_path = staged_uri
312         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
313         db.add_video(
314             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
315         )
316         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
317         if not out_root:
318             return _failed(
319                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
320                 "? that's where the Cloud Run job writes the result.",
321                 False,
322             )
323
324         notify("dispatching (cloud_run)")
325         logger.info(
326             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
327             req.video_path,
328             out_root,
329         )
330         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
331         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
332         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
333         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
334         overrides = []
335         sk = getattr(getattr(cfg, "indexing", None), "skip_ai_handoff", None)
336         if sk is not None:
337             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
338         spec = ComputeJobSpec(
339             video_uri=req.video_path,
340             out_uri=out_root,
341             segmenter=seg_name,
342             config_overrides=tuple(overrides),
343         )
344         try:
345             result = compute.run_infer(
346                 spec
347             ) # dispatch the Cloud Run Job + bucket-poll to completion
348         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
349             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
350             return _failed(f"cloud dispatch error: {e}", True)
351         if result.returncode != 0:
352             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
353         if result.video_id and result.video_id != video_id:
354             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
355             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.

```



```

356         logger.warning(
357             "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
358             "and the worker hash; ingesting under the local id.",
359             result.video_id,
360             video_id,
361         )
362
363     notify("ingesting (cloud_run)")
364     storage_dict = (
365         storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
366         if hasattr(storage_cfg, "model_dump")
367         else {}
368     )
369     try:
370         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
371     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
372         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
373         return _failed(
374             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
375             True,
376         )
377
378     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
379     segments = [
380         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
381     ]
382     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
383     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
384     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
385     provenance = {
386         "path": "cloud_run",
387         "requested": req.compute,
388         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
389         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
390         "execution": result.execution,
391         "outputs_uri": result.outputs_uri,
392     }
393     logger.info(
394         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
395         result.execution,
396         provenance["job"],
397         provenance["region"],
398         result.outputs_uri,
399         video_id,
400     )
401     return (
402         {
403             "video_id": video_id,
404             "status": "success",
405             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
406             "results": [r.model_dump() for r in val_results],
407             "play_segments": segments,
408             "compute": provenance,
409         },
410         True,
411     )
412

```

```

413
414 def _execute_processing(
415     config=None, db=None, req=None, progress=None
416 ) -> Dict[str, Any]:
417     """The full hash ? validate ? segment ? report pipeline for one video.
418
419     Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
420     module-level ``global_config``/``db_instance`` (backward-compatible with existing
421     tests that pass only ``req``). Route handlers always pass explicit params; older
422     test code that calls ``_execute_processing(req)`` still works.
423
424     ``progress`` is an optional callable(stage: str) used to surface coarse job
425     progress.
426
427     Raises on unexpected internal errors ? the caller records those as a job failure
428     """
429     import backend.main as _main # call-time seam: main's globals/model stay patchable
430
431     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
432     # shift positional args ? req arrives in position 0 (config), progress in position 1
433     (db).
434     if config is not None and not isinstance(config, AppConfig):
435         # Old signature: _execute_processing(req, progress=None)
436         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
437             req = db
438             db = None
439         elif req is None and isinstance(config, _main.ProcessRequest):
440             req = config
441             config = None
442     # Resolve config/db from module globals when not passed explicitly
443     if config is None:
444         config = _main.global_config
445     if db is None:
446         db = _main.db_instance
447
448     notify = progress or (lambda stage: None)
449
450     notify("hashing")
451     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
452     (identity ?
453     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
454     original
455     # req.video_path (the source URI) is kept for the DB record + report; only the file-
456     reading
457     # consumers (hash / validators / segmenter) use the resolved local path.
458     local_video = ""
459     try:
460         local_video = storage.acquire_local_input(
461             req.video_path, getattr(config, "storage", None)
462         )
463         video_id = _main.compute_video_id(local_video)
464     except Exception:
465         storage.cleanup_acquired_local_input(
466             req.video_path, local_video, getattr(config, "storage", None)
467         )
468         raise
469     try:
470         existing_video = db.get_video(video_id)
471     except Exception:
472         storage.cleanup_acquired_local_input(
473             req.video_path, local_video, getattr(config, "storage", None)
474         )
475         raise
476     was_reingest = existing_video is not None
477
478     # typed AppConfig: attribute access for the default segmenter (with safe fallback)

```

```

473     default_seg = getattr(
474         getattr(config, "indexing", None), "default_segementer", "motion"
475     )
476     seg_name = req.segmenter or default_seg
477     segmenter_actual = None
478     segmentation_ran = False
479     val_results: List[Any] = []
480     val_context: Dict[str, Any] = {}
481     # Compute-path PROVENANCE (the validation method): which backend actually ran the
DETECT ?
482     # "in_process" once the local segmenter is reached, set on the response as
"cloud_run" by
483     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
484     compute_actual: Optional[str] = None
485     response: Optional[Dict[str, Any]] = None
486     try:
487         if (
488             existing_video
489             and existing_video.get("status") == "processed"
490             and not req.force
491         ):
492             cached_segments = db.query_play_segments(video_id, {})
493             if cached_segments:
494                 logger.info(
495                     "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
496                     video_id,
497                 )
498                 response = {
499                     "video_id": video_id,
500                     "status": "success",
501                     "message": (
502                         f"Reused {len(cached_segments)} cached play segments. "
503                         "Pass force=true to reprocess this video."
504                     ),
505                     "results": existing_video.get("validation_results", []),
506                     "play_segments": cached_segments,
507                     "cached": True,
508                     "compute": {
509                         "path": "cached",
510                         "requested": getattr(req, "compute", None),
511                     },
512                 }
513             return response
514
515     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report)
516     notify("validating")
517     logger.info(f"Running validations for {req.video_path}...")
518     val_results, val_context = registry.run_all_with_context(local_video, config)
519     failures = [r for r in val_results if not r.passed]
520
521     if failures:
522         # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
523         # status; it no longer destroys the video's prior good segments.
524         db.add_video(
525             video_id,
526             req.video_path,
527             "failed",
528             [r.model_dump() for r in val_results],
529         )
530         response = {
531             "video_id": video_id,
532             "status": "failed",

```

```

533         "message": _validation_failure_message(failures),
534         "results": [r.model_dump() for r in val_results],
535     }
536     return response
537
538     # 2. Run segmenter & indexer
539     logger.info("[API] Video passed checks. Segmenting and indexing...")
540     db.add_video(
541         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
542     )
543
544     segmenter_cls = segmenter_registry.get(seg_name)
545     if not segmenter_cls:
546         # Submit-time validation already 400s unknown names; this is the defensive
547         # in-worker path (e.g. config default pointing at an unregistered
548         segmenter).
549         available = list(segmenter_registry.list_available().keys())
550         response = {
551             "video_id": video_id,
552             "status": "failed",
553             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
554 {available}",
555             "results": [r.model_dump() for r in val_results],
556             "play_segments": [],
557         }
558         return response
559
560     logger.info(f"[API] Using segmenter: {seg_name}")
561     notify(f"segmenting ({seg_name})")
562     segmenter_actual = seg_name
563     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
564     only
565     # if the run yields segments; otherwise drop the new partial rows and keep the
566     old.
567     play_wm, cand_wm = db.segment_watermarks(video_id)
568     # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
569     run the heavy
570     # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
571     unchanged.
572     # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
573     segmenter below runs
574     # byte-identically. (player_pool/max_frames aren't threaded to the worker
575     `infer` CLI yet, so
576     # they're dropped on the cloud path in this slice ? a documented follow-up.)
577     remote = _maybe_dispatch_remote(
578         config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
579     )
580     if remote is not None:
581         response, segmentation_ran = (
582             remote # `ran` = did the cloud job actually execute (report)
583         )
584         return response # the cloud branch already stamped response["compute"]
585     (path=cloud_run)
586     # In-process from here on ? record the actual path for the provenance stamp in
587     `finally`.
588     compute_actual = "in_process"
589     segmenter = segmenter_cls(db, config)
590     try:
591         # Serialize the CPU/GPU-heavy in-process detection (see
592         _LOCAL_COMPUTE_LANE). A remote
593         # dispatch never reaches here (it returned above), so remote polls still
594         overlap freely.
595         with _LOCAL_COMPUTE_LANE:
596             result = segmenter.process_video(
597                 video_id, local_video, req.player_pool, req.max_frames

```

```

586         )
587     except Exception:
588         # A raising segmenter must not leave half-written rows: restore the pre-run
589         # state (same destroy-after-confirm contract as the Failure branch), then
590         let
591             # the job worker record the failure.
592             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
593             raise
594     segmentation_ran = True
595
596     if isinstance(result, Failure):
597         logger.error(f"[API] Segmenter failed: {result.message}")
598         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
599         response = {
600             "video_id": video_id,
601             "status": "failed",
602             "message": f"Segmenter '{seg_name}' failed: {result.message}",
603             "results": [r.model_dump() for r in val_results],
604             "play_segments": [],
605         }
606         return response
607
608     segments = result.value if hasattr(result, "value") else result
609     notify("finalizing")
610     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
611
612     response = {
613         "video_id": video_id,
614         "status": "success",
615         "message": f"Successfully indexed {len(segments)} play segments using
616         '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
617         .value
618         "results": [r.model_dump() for r in val_results],
619         "play_segments": segments,
620     }
621     return response
622
623     finally:
624         # Compute-path PROVENANCE stamp (the validation method): the cloud branch
625         already set
626         # response["compute"] (path=cloud_run + the Cloud Run execution); for every
627         other path stamp
628         # the actual path here so the job result PROVES which backend ran (in_process /
629         not_run) and
630         # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
631         then visible.
632         if isinstance(response, dict):
633             response.setdefault(
634                 "compute",
635                 {
636                     "path": compute_actual or "not_run",
637                     "requested": getattr(req, "compute", None),
638                 },
639             )
640         prov = response.get("compute", {}) if isinstance(response, dict) else {}
641         logger.info(
642             "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
643             video_id,
644             getattr(req, "compute", None),
645             prov.get("path"),
646             prov.get("dispatched"),
647             (response or {}).get("status") if isinstance(response, dict) else None,
648         )
649         # Always emit the per-video observability report (next to the video, or to
650         # report.output_dir). Never raises. Surface the paths in the response. (The
651         compute-path

```

```

643         # proof lives on the job result + the [COMPUTE] log, not the report ?
obsreport's record_run
644         # doesn't surface arbitrary run signals.)
645         paths = _main.emit_indexer_report(
646             db=db,
647             video_id=video_id,
648             # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
the
649             # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
local://)
650             # collapses to a literal scheme dir at the repo root (see #397). The source
URI is
651             # still preserved in DB provenance via db.add_video.
652             video_path=local_video,
653             config=_report_config_for_input(config, req.video_path),
654             val_results=val_results,
655             val_context=val_context,
656             segmenter_requested=req.segmenter,
657             segmenter_actual=segmenter_actual,
658             was_reingest=was_reingest,
659             segmentation_ran=segmentation_ran,
660         )
661         if paths and isinstance(response, dict):
662             response["reports"] = paths
663         storage.cleanup_acquired_local_input(
664             req.video_path, local_video, getattr(config, "storage", None)
665         )
666
667
668     def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
669         """Best-effort size (bytes) of a remote storage source, or ``None`` if unknowable.
670
671         Used only to pre-reserve scratch for a fetch-to-local (P2). Never raises: a stat
that needs a
672         missing cloud SDK, the network, or a credential returns ``None`` so the free-space
guard degrades
673         to a no-op rather than blocking a job it cannot measure."""
674         try:
675             sc = (
676                 storage_config.model_dump()
677                 if hasattr(storage_config, "model_dump")
678                 else (storage_config or {})
679             )
680             backend = storage.get_storage(input_ref.backend, config=sc)
681             return int(backend.stat(input_ref).size)
682         except Exception as e: # noqa: BLE001 - measuring is best-effort; inability must
not block
683             logger.debug(
684                 "free-space: could not stat remote source %s: %s", input_ref.uri, e
685             )
686             return None
687
688
689     class _CompileError(Exception):
690         """A submit-time-validatable compile problem ? carries the HTTP status + detail so
the API path
691         maps it to an HTTPException (fast 4xx) and the worker path maps it to a failed-job
result."""
692
693         def __init__(self, status_code: int, detail: str):
694             super().__init__(detail)
695             self.status_code = status_code
696             self.detail = detail
697
698

```

```

699     def _resolve_compile(
700         db, cfg, video_id: str, segment_ids: List[int], output_filename: str
701     ):
702         """Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
703         resolve, since a
704         row could change between submit and run). Verifies the video exists, the requested
705         segments match
706         and survive the ``stitching.min_shot_count`` floor (segments WITHOUT a known
707         shot_count are exempt
708         so manual/legacy picks can't silently vanish), and the output name is traversal-
709         safe. Raises
710         ``_CompileError(404/400)``. Returns ``(video_row, kept_segments,
711         safe_out_name)``."""
712         video = db.get_video(video_id)
713         if not video:
714             raise _CompileError(404, "Indexed video record not found.")
715         # Reject path traversal / absolute names (os.path.join would otherwise write
716         anywhere on disk).
717         try:
718             out_name = safe_output_filename(output_filename)
719         except ValueError as e:
720             raise _CompileError(400, str(e)) from e
721
722         all_segments = db.query_play_segments(video_id, {})
723         matched = [s for s in all_segments if s["id"] in segment_ids]
724         if not matched:
725             raise _CompileError(400, "No matching play segments found to stitch.")
726
727         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
728         kept = []
729         for s in matched:
730             meta = s.get("metadata") or {}
731             sc = meta.get("shot_count") if isinstance(meta, dict) else None
732             try:
733                 if sc is not None and int(sc) < stitching.min_shot_count:
734                     continue
735             except (TypeError, ValueError):
736                 pass # unparseable count -> treat as unknown, keep
737             kept.append(s)
738         if len(kept) < len(matched):
739             logger.info(
740                 "[compile] stitching.min_shot_count=%s: dropped %d/%d segments below the
741                 floor.",
742                 stitching.min_shot_count,
743                 len(matched) - len(kept),
744                 len(matched),
745             )
746         if not kept:
747             raise _CompileError(
748                 400,
749                 f"All {len(matched)} matching segments fall below "
750                 f"stitching.min_shot_count={stitching.min_shot_count}.",
751             )
752         return video, kept, out_name
753
754     def reel_object_uri(cfg, video_id: str, name: str) -> str:
755         """The bucket URI a compiled reel is mirrored to for a direct signed-URL download
756         (#463), or "" when
757         the output isn't a cloud bucket (truly-local ? serve the local file). Shared by the
758         compile
759         mirror-upload and the /api/highlights redirect:
760         ``<output_root>/highlights/<video_id>/<name>``."""
761         root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
762         if not root or "://" not in root:

```

```

754         return ""
755     return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
756
757
758     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
759         """The active storage settings dict (per-backend config for the storage layer ? S3
760 endpoint/region,
761         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
762 default backend
763         isn't silently constructed as a fresh default."""
764         sc = getattr(cfg, "storage", None)
765         return sc.model_dump() if sc is not None else None
766
767     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
768         """A short-lived signed URL for a reel mirrored to the output bucket, so
769 /api/highlights can
770         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
771 ephemeral local
772         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
773 config for BOTH
774         the existence check and the signing (extracted here to keep backend/main.py lean ?
775 P23/P24)."""
776         reel_uri = reel_object_uri(cfg, video_id, name)
777         if not reel_uri:
778             return None
779         settings = _storage_settings(cfg)
780         if storage.exists(reel_uri, config=settings):
781             return storage.signed_url(reel_uri, config=settings, method="GET")
782         return None
783
784     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
785         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
786 stitch the reel,
787         and return the {status, filepath, download_url} result the SPA polls for. Returns a
788         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
789 stitch error)
790         so the SPA always gets a clean verdict instead of a worker-crash error."""
791         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
792 backend.main
793
794         notify = progress or (lambda stage: None)
795         params = json.loads(job.get("params") or "{}")
796         video_id = job.get("video_id") or ""
797         segment_ids = params.get("segment_ids") or []
798         locale = params.get("locale")
799
800         notify("resolving")
801         try:
802             video, kept, out_name = _resolve_compile(
803                 db, cfg, video_id, segment_ids, params.get("output_filename", "")
804             )
805         except _CompileError as e:
806             return {"status": "failed", "message": e.detail, "video_id": video_id}
807
808         stitching = getattr(cfg, "stitching", None) or StitchingConfig()
809         output_dir = (
810             getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
811         )
812         # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
813 the job);
814         # unknown/absent locale keeps the configured default unchanged.
815         localized_watermark = localize_watermark(stitching.watermark, locale)
816         compiler = _main.VideoCompiler(

```



```

809         output_dir,
810         begin_padding=stitching.begin_padding,
811         end_padding=stitching.end_padding,
812         watermark=localized_watermark,
813     )
814     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
815
816     notify("stitching")
817     local_src = ""
818     try:
819         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
820         # path). Resolve to
821         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
822         # scratch for a URI (the
823         # same seam the process path uses); without it, compiling a bucket-ingested
824         video can't open gs://.
825         local_src = storage.acquire_local_input(
826             video["filepath"], getattr(cfg, "storage", None)
827         )
828         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
829         (for remote
830         # dispatch) doesn't run multiple compiles at once on the box (see
831         _LOCAL_COMPUTE_LANE).
832         with _LOCAL_COMPUTE_LANE:
833             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
834     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
835         FAILURE, not a crash
836         logger.error(
837             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
838         )
839         return {
840             "status": "failed",
841             "message": f"Highlight stitching failed: {e}",
842             "video_id": video_id,
843         }
844     finally:
845         if local_src:
846             storage.cleanup_acquired_local_input(
847                 video["filepath"], local_src, getattr(cfg, "storage", None)
848             )
849
850     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
851     URL ? Cloud Run
852     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
853     copy dies when the
854     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
855     (the endpoint
856     # still falls back to the local file on the same instance).
857     reel_uri = reel_object_uri(cfg, video_id, out_name)
858     if reel_uri:
859         try:
860             notify("uploading reel")
861             # Thread the active storage settings so the configured backend (S3
862             endpoint/region, GCS
863             # project/client) is used, not a fresh default.
864             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
865         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
866             already succeeded
867             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
868
869     # `filepath` is server-local (not browser-reachable); also return the download URL
870     so the SPA needn't
871     # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
872     download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
873     notify("finished")

```

```

862         return {
863             "status": "success",
864             "filepath": out_path,
865             "download_url": download_url,
866             "video_id": video_id,
867         }
868

```

9. user (2026-07-04T11:49:14.586Z)

```

1     import argparse
2     import json
3     import logging
4     import os
5     import shutil
6     import sys
7     import tempfile
8     import threading
9     import uuid
10    from importlib.metadata import PackageNotFoundError
11    from importlib.metadata import version as _pkg_version
12    from urllib.parse import quote
13
14    import uvicorn
15    from dotenv import load_dotenv
16
17    # PACKAGING_PLAN.md P0a: prefer an app-home .env ONLY when RALLY_APP_HOME is set (the
packaged
18    # launcher pins it); otherwise env_file_to_load() returns None and load_dotenv(None)
keeps the
19    # exact historical frame-relative walk-up ? byte-identical to the pre-P0a bare
load_dotenv().
20    # stdlib-only import ? keeps this pre-pydantic line light.
21    from backend.utils.app_home import env_file_to_load, resolve_output_dir
22
23    load_dotenv(env_file_to_load())
24
25    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
26    logging.basicConfig(
27        level=logging.INFO,
28        format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
29        datefmt="%H:%M:%S",
30    )
31    logger = logging.getLogger(__name__)
32    from typing import Any, Dict, List, Optional
33
34    from fastapi import FastAPI, HTTPException, Request
35    from fastapi.middleware.cors import CORSMiddleware
36    from fastapi.responses import (
37        FileResponse,
38        HTMLResponse,
39        JSONResponse,
40        RedirectResponse,
41    )
42    from fastapi.staticfiles import StaticFiles
43    from pydantic import BaseModel
44    from starlette.middleware.trustedhost import TrustedHostMiddleware
45
46    from backend import (
47        billing, # M8: per-job cost ledger (USD internal / INR display)
48        storage, # M2: URI-aware input acquisition (local = identity passthrough)
49    )
50    from backend.api import (
51        auth as edge_auth, # M9: Cloudflare Access edge-auth (default-OFF)

```

```

52     )
53     from backend.config import (
54         AppConfig,
55         StartupCheckError,
56         enforce_startup_checks,
57         write_light_default_config, # P2b: batteries-included first-run config seeding
58     )
59     from backend.config import ( # new typed config
60         load_config as load_app_config,
61     )
62     from backend.infrastructure.database import Database
63     from backend.pipeline.segmenters.base import segmenter_registry
64     from backend.pipeline.stitcher.compiler import VideoCompiler
65     from backend.pipeline.validators.base import registry
66     from backend.providers import provider_registry
67     from backend.reporting import emit_indexer_report
68     from backend.results import Failure # standardized error/result contract
69
70     # Server/network bootstrap helpers (CORS/host-allowlist config + the `indexer --app`
71     # launcher)
72     # live in backend.server_runtime (P24 god-module split). Imported HERE, above `app =
73     # FastAPI(...)`
74     # because the module-level add_middleware(...) below calls
75     # _cors_origins_from_env/_allowed_hosts_from_env
76     # at import time; re-exported so `import backend.main as m; m._resolve_bind_host(...)`
77     # keeps working.
78     from backend.server_runtime import ( # noqa: F401 (re-exports: names stay addressable
79     on backend.main)
80         _allowed_hosts_from_env,
81         _cors_origins_from_env,
82         _find_free_port,
83         _open_browser_when_ready,
84         _port_is_free,
85         _resolve_app_port,
86         _resolve_bind_host,
87     )
88     from backend.storage.capabilities import (
89         storage_capabilities, # P3: secret-free "which media can this box use" report
90     )
91     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
92         ffmpeg_bin,
93         ffprobe_bin,
94     )
95     from backend.utils.freespace import require_free_bytes # HTTP-507 disk guard (P2, D11)
96     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
97     from backend.utils.paths import (
98         output_base,
99         resolve_under_root,
100         safe_output_filename,
101         validate_input_path,
102     )
103     from backend.utils.redact import (
104         redact_secrets, # P0b: mask secret-shaped fields in /api/config
105     )
106     from backend.utils.single_instance import acquire_lock # P0e: one instance per database
107
108     app = FastAPI(title="Sports Highlight Indexer API")
109
110     def create_app(config: AppConfig, db: Database) -> FastAPI:
111         """Factory: inject shared state via ``app.state`` so route handlers access it
112         through
113         ``request.app.state`` ? eliminating the Optional module-level globals that forced
114         ``backend.main`` and ``backend.api.job_worker`` into mypy quarantine.
115     """

```

```

111     Also sets the module-level ``global_config`` / ``db_instance`` for transitional
112     compatibility (helpers and tests still reference them). Route handlers use
113     ``request.app.state`` exclusively."""
114     global global_config, db_instance
115     global_config = config
116     db_instance = db
117     app.state.config = config
118     app.state.db = db
119     return app
120
121
122     # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
123     browsers per the
124     # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
125     credentials stay
126     # off. The origin list is configurable via the env var above (default [""]) so a hosted
127     deploy can
128     # lock it to its origin; the middleware is fixed at startup.
129     app.add_middleware(
130         CORSMiddleware,
131         allow_origins=_cors_origins_from_env(),
132         allow_credentials=False,
133         allow_methods=["*"],
134         allow_headers=["*"],
135     )
136
137     # DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are
138     honoured
139     # (a remote page can't read responses cross-origin AND its Host header is rejected
140     here). www_redirect
141     # off so an unexpected ``www.`` host is rejected rather than 307-redirected.
142     app.add_middleware(
143         TrustedHostMiddleware,
144         allowed_hosts=_allowed_hosts_from_env(),
145         www_redirect=False,
146     )
147
148     # --- P2: the friendly `indexer --app` launcher (cross-platform, no native component ?
149     01) ---
150
151     # Serves static frontend files directly (now under api/ per approved structure).
152     # PACKAGING_PLAN.md P0a: resolve PACKAGE-relative (not CWD-relative) so launching from
153     an
154     # arbitrary directory finds the shipped assets instead of creating a stray
155     ./backend/api/static.
156     # (The legacy in-engine dashboard; the real SPA gets bundled here in P1a.)
157     _STATIC_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "api", "static")
158     try:
159         os.makedirs(_STATIC_DIR, exist_ok=True)
160     except OSError:
161         # The dir ships with the package (a no-op create); guard against a read-only install
162         # tree (system-installed Python) since exist_ok suppresses FileExistsError, not
163         PermissionError.
164         pass
165     app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
166
167     # P1b: serve the bundled KhelSutra SPA single-origin. `backend/ui/` is the committed
168     snapshot
169     # (produced by scripts/bundle_ui.sh); fall back to the legacy in-engine dashboard only
170     if it's
171     # absent (a source checkout with no vendored UI). The SPA calls same-origin `/api`, so
172     no SPA change

```

```

164     # is needed; it is query-param driven (?video=?), so serving index.html at `/' + /assets
is enough
165     # (no client-side path routing ? no history-fallback catch-all, which would also be a
route-order hazard).
166     _BUNDLED_UI_DIR = os.path.join(os.path.dirname(os.path.abspath(__file__)), "ui")
167     _UI_DIR = (
168         _BUNDLED_UI_DIR
169         if os.path.isfile(os.path.join(_BUNDLED_UI_DIR, "index.html"))
170         else _STATIC_DIR
171     )
172     _UI_ASSETS_DIR = os.path.join(_UI_DIR, "assets")
173     if os.path.isdir(_UI_ASSETS_DIR):
174         app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
175
176
177     @app.get("/", response_class=HTMLResponse)
178     def serve_index():
179         index_path = os.path.join(_UI_DIR, "index.html")
180         if os.path.exists(index_path):
181             with open(index_path, "r", encoding="utf-8") as f:
182                 return f.read()
183         return "<h3>Sports Highlight Indexer Server is Running. (No bundled UI found.)</h3>"
184
185
186     def _bundled_ui_version() -> Optional[Dict[str, Any]]:
187         """Provenance of the vendored SPA (backend/ui/ui_version.json), or None if no UI is
bundled.
188         Includes `engine_api_version` ? the API_VERSION the snapshot was built against (skew
guard)."""
189         path = os.path.join(_BUNDLED_UI_DIR, "ui_version.json")
190         try:
191             with open(path, "r", encoding="utf-8") as f:
192                 return json.load(f)
193         except (OSError, ValueError):
194             return None
195
196
197     # Global variables initialized at startup
198     db_instance: Optional[Database] = None
199     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
200     # P0e: the single-instance lock handle. Kept module-global for the process lifetime ? if
it were
201     # GC'd/closed the OS would release the lock. None until main() acquires it (server/CLI
init).
202     _instance_lock: Optional[Any] = None
203
204     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
205     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
206     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
207     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
208     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
209     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
210     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
211     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
212         _MAX_DRAIN_WAKE_SEC,
213         JobWaiting,
214         _arm_wake_timer,
215         _drain_due_jobs,
216         _get_executor,
217         _job_to_request,
218         _maybe_record_job_cost,
219         _run_claimed_job,

```

```

220     _schedule_next_drain_if_waiting,
221     configure_drain,
222 )
223
224 # The processing/compile ORCHESTRATION (hash?validate?segment?report pipeline, Cloud Run
225 # dispatch+ingest, compile resolution + worker handler) now lives in
backend.orchestration
226 # (P23 god-module split; re-cut of #390). Re-exported here so the names stay addressable
on
227 # backend.main ? `import backend.main as m; m._execute_processing(...)` the job
worker's
228 # call-time `_main._execute_compile(...)` seam, and monkeypatch.setattr(backend.main,
...)
229 # all keep working unchanged.
230 from backend.orchestration import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
231     _cloud_available,
232     _CompileError,
233     _execute_compile,
234     _execute_processing,
235     _finalize_reingest,
236     _ingest_remote_segments,
237     _maybe_dispatch_remote,
238     _remote_input_size_bytes,
239     _report_config_for_input,
240     _resolve_compile,
241     signed_reel_url,
242 )
243
244
245 # Legacy thin wrapper for any remaining direct calls during transition.
246 # New code should import load_app_config / AppConfig from backend.config directly.
247 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
248     cfg = load_app_config(config_path)
249     return cfg.model_dump(mode="json")
250
251
252 # --- API Models ---
253 class ProcessRequest(BaseModel):
254     video_path: str
255     player_pool: Optional[List[str]] = None
256     max_frames: Optional[int] = None
257     segmenter: Optional[str] = None
258     # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
259     # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
260     locale: Optional[str] = None
261     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
262     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
263     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
264     compute: Optional[str] = None
265     # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
266     # play_segments is returned from cache so refresh/Try again cannot re-bill paid
segmenters.
267     force: bool = False
268
269
270 class AnnotationsRequest(BaseModel):
271     # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio

```

```

272         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
273         video_id: str
274         csv: str
275
276
277     class AssetRequest(BaseModel):
278         # "Store this video in my medium" (STUDIO_MEDIA_LIFECYCLE.md P4, D14). Exactly ONE
source:
279         #     uploaded_path ? a local file from the chunked upload (the durable copy's source;
kept as the
280         #         local working copy for the run), OR
281         #     source_uri ? a video ALREADY at rest in a medium (a pasted
s3:///gcs:///gdrive:// URI) ?
282         #         registered as-is, no copy (D14b).
283         # medium_uri is the destination store (a bucket/Drive/local folder URI). The asset
is keyed by
284         # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content
hash.
285         uploaded_path: Optional[str] = None
286         source_uri: Optional[str] = None
287         medium_uri: str
288         sport: Optional[str] = None
289
290
291     class QueryRequest(BaseModel):
292         video_id: str
293         server: Optional[str] = None
294         receiver: Optional[str] = None
295         duration_min: Optional[float] = None
296         event_type: Optional[str] = None
297
298
299     class CompileRequest(BaseModel):
300         video_id: str
301         segment_ids: List[int]
302         output_filename: str
303         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Localizes the reel
watermark
304         # (text + script font). Optional ? absent/unknown keeps the configured default (en
behavior).
305         locale: Optional[str] = None
306
307
308     # --- API Endpoints ---
309     @app.get("/api/config")
310     def get_config(request: Request):
311         # P0b: this endpoint is unauthenticated, so never serialize a credential-shaped
field. Today
312         # AppConfig holds no secrets (the Gemini key is env-only; cf_access_aud is a public
id), so this
313         # is byte-identical for the real fields ? but it fails-closed if a secret is ever
added (guarded
314         # by tests/test_redact.py + the /api/config redaction test).
315         cfg = request.app.state.config
316         if cfg is None:
317             return {}
318         return redact_secrets(cfg.model_dump(mode="json"))
319
320
321     @app.get("/api/health")
322     def health(request: Request):
323         """Cheap liveness/readiness probe (no auth) for the SPA's offline banner
324         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
confirm the engine is reachable and which detector is wired. Never raises."""
325

```

```

326     cfg = request.app.state.config
327     sport = getattr(cfg, "sport", None)
328     indexing = getattr(cfg, "indexing", None)
329     return {
330         "status": "ok",
331         "sport": sport,
332         "default_segmenter": getattr(indexing, "default_segmenter", None),
333     }
334
335
336     # P1: the bundled SPA pins this ? the contract version of the /api surface. BUMP when an
existing
337     # endpoint's request/response shape changes incompatibly (purely additive endpoints need
no bump).
338     API_VERSION = 1
339
340
341     def _engine_version() -> str:
342         """Installed package version (wheel/editable); 'unknown' if not importable as a
distribution."""
343         try:
344             return _pkg_version("badminton-highlight-indexer")
345         except PackageNotFoundError:
346             return "unknown"
347
348
349     @app.get("/api/version")
350     def api_version():
351         """Version handshake so a bundled SPA can detect engine skew (PACKAGING_PLAN.md
P1)."""
352         return {
353             "engine_version": _engine_version(),
354             "api_version": API_VERSION,
355             "ui": _bundled_ui_version(),
356         }
357
358
359     @app.get("/api/capabilities")
360     def capabilities(request: Request):
361         """Honest probe of what THIS box can do ? the first-run wizard renders the compute
step from
362         it (PACKAGING_PLAN.md P1). No secrets: reports only whether a Gemini key is PRESENT,
never its
363         value. Best-effort + never raises (the wizard degrades gracefully)."""
364         cfg = request.app.state.config
365         indexing = getattr(cfg, "indexing", None)
366         deployment = getattr(cfg, "deployment", None)
367         ffmpeg = shutil.which(ffmpeg_bin()) # P2a: honor RALLY_FFMPEG / RALLY_FFPROBE
368         ffprobe = shutil.which(ffprobe_bin())
369         # WASB weights are best-effort: env first (matches native_wasb_runner), then config.
370         wasb = getattr(indexing, "wasb", None)
371         weights_path = (
372             os.environ.get("WASB_WEIGHTS") or getattr(wasb, "weights_path", "") or ""
373         )
374         # Cheap GPU heuristic: nvidia-smi on PATH (matches scripts/doctor.py). Deliberately
NOT
375         # torch.cuda.is_available() ? torch is absent in the LIGHT tier (so it would false-
negative on a
376         # real GPU box) and importing it would add seconds to this probe. The detector
healthcheck is the
377         # authority on whether GPU inference actually works; this is just the wizard's
first-order signal.
378         nvidia_smi = shutil.which("nvidia-smi")
379         return {
380             "api_version": API_VERSION,

```



```

381     "engine_version": _engine_version(),
382     "segmenters": sorted(segmenter_registry.list_available().keys()),
383     "default_segmenter": getattr(indexing, "default_segmenter", None),
384     "ffmpeg": {
385         "ffmpeg": bool(ffmpeg),
386         "ffprobe": bool(ffprobe),
387         "ffmpeg_path": ffmpeg,
388         "ffprobe_path": ffprobe,
389     },
390     "gpu": {"nvidia_smi_present": bool(nvidia_smi)},
391     "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
392     "wasb_weights_present": bool(weights_path) and os.path.exists(weights_path),
393     "deployment": {
394         "profile": getattr(deployment, "profile", "") or "",
395         "compute_target": getattr(deployment, "compute_target", "") or "",
396         # item 3: can this server satisfy a per-request compute="cloud" dispatch?
397         # the "run in the cloud" toggle only when true (else it's local-only).
398         "cloud_available": _cloud_available(cfg),
399     },
400     # P3: which storage media THIS box can store INTO (local always; s3/gcs if
401     # gdrive if a mount resolves). The P5 medium picker renders only usable
402     # SDK+config;
403     "storage": storage_capabilities(
404         getattr(cfg, "storage", None), local_free_path=output_base(cfg)
405     ),
406 }
407
408 def _friendly_video_name(filepath: str) -> str:
409     """A human name for a video from its stored path/URI ? the filename stem
410     (``?/match_2026.mp4`` ?
411     ``match_2026``). Works for a local path, a bucket key, or a ``gdrive://`` URI
412     (rsplit on ``/``).
413     Falls back to the whole ``filepath`` if there's no basename."""
414     base = str(filepath or "").rstrip("/").rsplit("/", 1)[-1].rsplit("\\", 1)[-1]
415     stem = os.path.splitext(base)[0]
416     return stem or base or str(filepath or "")
417
418 def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
419     """Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
420     None)``. The
421     ``resolution_fps_check`` validator already probed ffprobe and stored
422     ``details={width,height,fps}``
423     on its result (every branch) ? so the videos row carries it with no schema change /
424     re-probe."""
425     for r in validation_results or []:
426         if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
427             d = r.get("details") or {}
428             w, h, fps = d.get("width"), d.get("height"), d.get("fps")
429             resolution = f"{w}x{h}" if w and h else None
430             return (fps if isinstance(fps, (int, float)) else None), resolution
431     return None, None
432
433 def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
434     """Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
435     ``id``/``filepath`` (an
436     existing test + any old client rely on them) and adds the fields the SPA library +
437     the persistent
438     "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
439     friendly
440     ``match_name`` (so an uploaded video shows its filename, not a generic fallback),

```

```

``local_path``,
435     ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
436     validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
437     real agree.""
438     fp = row.get("filepath") or ""
439     fps, resolution = _video_dims(row.get("validation_results"))
440     return {
441         **row,
442         "video_id": row.get("id"),
443         "match_name": _friendly_video_name(fp),
444         "local_path": fp,
445         "created_at": row.get("processed_at") or row.get("created_at"),
446         "fps": fps,
447         "resolution": resolution,
448     }
449
450
451     @app.get("/api/videos")
452     def list_videos(request: Request, limit: Optional[int] = None):
453         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
454         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
455         (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
456         names and the persistent "current video" label survives an upload ? see
``_video_summary``.""
457         return {
458             "videos": [
459                 _video_summary(v) for v in request.app.state.db.list_videos(limit=limit)
460             ]
461         }
462
463
464     def _reels_dir(cfg) -> str:
465         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
466
467
468     _REEL_EXTS = (".mp4", ".mov")
469
470
471     @app.get("/api/reels")
472     def list_reels(request: Request):
473         """List compiled highlight reels present in the output dir, newest first, with a
download URL.
474
475         NB: reels are stored FLAT in output_dir under the bare name passed to /api/compile ?
the engine
476         does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
477         this is a global list. (Per-video reel tracking would need a DB record + compile
change ? a
478         later slice.) Only top-level video files are listed (per-video scratch subdirs are
skipped)."""
479         base = _reels_dir(request.app.state.config)
480         reels = []
481         try:
482             entries = os.scandir(base)
483         except OSError:
484             return {"reels": []}
485         with entries:
486             for e in entries:
487                 if e.is_file() and e.name.lower().endswith(_REEL_EXTS):
488                     try:

```

```

489         st = e.stat()
490     except OSError:
491         continue
492     reels.append(
493         {
494             "filename": e.name,
495             "size_bytes": st.st_size,
496             "modified_at": st.st_mtime,
497             "download_url": f"/api/reels/{quote(e.name)}",
498         }
499     )
500     reels.sort(key=lambda r: r["modified_at"], reverse=True)
501     return {"reels": reels}
502
503
504 @app.get("/api/reels/{filename}")
505 def download_reel(filename: str, request: Request):
506     """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
507     Mirrors /api/highlights' safety (bare-name + containment checks)."""
508     try:
509         name = safe_output_filename(filename)
510     except ValueError as e:
511         raise HTTPException(status_code=400, detail=str(e)) from e
512     base = _reels_dir(request.app.state.config)
513     path = os.path.join(base, name)
514     try:
515         path = resolve_under_root(path, base)
516     except ValueError as e:
517         raise HTTPException(status_code=400, detail=str(e)) from e
518     if not os.path.isfile(path) or not name.lower().endswith(_REEL_EXTS):
519         raise HTTPException(status_code=404, detail=f"No compiled reel named '{name}'")
520     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
521     return FileResponse(path, media_type=media, filename=name)
522
523
524 # --- Annotate-in-UI (P8): serve a video's source + persist finished rally labels
-----
525
526
527 # In-flight annotation-proxy generations ? dedup so a browser's many Range requests to
/api/source
528 # don't each kick off a transcode.
529 _proxy_inflight: "set[str]" = set()
530 _proxy_lock = threading.Lock()
531 _source_cache_locks: Dict[str, threading.Lock] = {}
532 _source_cache_locks_guard = threading.Lock()
533
534
535 def _proxies_dir(cfg) -> str:
536     return os.path.join(output_base(cfg), "proxies")
537
538
539 def _source_cache_dir(cfg) -> str:
540     return os.path.join(output_base(cfg), "source_cache")
541
542
543 def _source_cache_lock(video_id: str) -> threading.Lock:
544     with _source_cache_locks_guard:
545         return _source_cache_locks.setdefault(video_id, threading.Lock())
546
547
548 def _source_cache_path(video_id: str, input_ref, cfg) -> str:
549     ext = os.path.splitext(input_ref.name or "")[1].lower()
550     if not ext or "/" in ext or "\\" in ext or len(ext) > 16:

```

```

551         ext = ".mp4"
552         cache_dir = _source_cache_dir(cfg)
553         name = safe_output_filename(f"{video_id}{ext}")
554         return resolve_under_root(os.path.join(cache_dir, name), cache_dir)
555
556
557     def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
558         """Fetch a remote source once into an output-scoped source/proxy cache."""
559         cache_path = _source_cache_path(video_id, input_ref, cfg)
560         if os.path.isfile(cache_path):
561             return cache_path
562         lock = _source_cache_lock(video_id)
563         with lock:
564             if os.path.isfile(cache_path):
565                 return cache_path
566             cache_dir = os.path.dirname(cache_path)
567             os.makedirs(cache_dir, exist_ok=True)
568             need = _remote_input_size_bytes(input_ref, getattr(cfg, "storage", None))
569             require_free_bytes(cache_dir, need, stage="source-cache")
570             tmp = os.path.join(
571                 cache_dir, f"{os.path.basename(cache_path)}.{uuid.uuid4().hex}.tmp"
572             )
573             try:
574                 storage.fetch(input_ref.uri, tmp, config=_storage_settings(cfg))
575                 os.replace(tmp, cache_path)
576             finally:
577                 try:
578                     if os.path.exists(tmp):
579                         os.remove(tmp)
580                 except OSError:
581                     pass
582             return cache_path
583
584
585     def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
586         """Background, best-effort: generate the annotation proxy ONCE so subsequent
587         /api/source loads
588         serve the lighter, seekable copy. Deduped; never blocks the request; the source
589         keeps serving if
590         it fails."""
591         with _proxy_lock:
592             if video_id in _proxy_inflight:
593                 return
594             _proxy_inflight.add(video_id)
595         try:
596             from backend.utils.annotation_proxy import make_annotation_proxy, needs_proxy
597
598             if not needs_proxy(src_local):
599                 return # already light enough ? no proxy worth making
600             make_annotation_proxy(
601                 src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
602             )
603         except Exception as e: # noqa: BLE001 - warming is best-effort
604             logger.warning("proxy: warm failed for %s: %s", video_id, e)
605         finally:
606             with _proxy_lock:
607                 _proxy_inflight.discard(video_id)
608
609     @app.get("/api/source/{video_id}")
610     def get_source(video_id: str, request: Request):
611         """Stream a video for in-UI annotation (P8), Range-capable so the browser can scrub.
612         Serves a
613         frame-preserving PROXY (STUDIO_MEDIA_LIFECYCLE.md D4 ? downscaled + seekable, labels
614         still keyed

```

```

612         to the master's MD5) once one exists; otherwise serves the source and WARMS a proxy
in the
613         background for next time ? non-blocking, so the first load is never delayed. Remote
sources are
614         fetched once into an output-scoped source cache, then served and proxied from that
stable copy. """
615         rec = request.app.state.db.get_video(video_id)
616         if not rec or not rec.get("filepath"):
617             raise HTTPException(status_code=404, detail="Video not found.")
618         cfg = request.app.state.config
619
620         # 1) A ready proxy wins. Bare-name + containment guarded like /api/highlights.
621         try:
622             proxy_name = safe_output_filename(f"{video_id}.mp4")
623             proxy_path = resolve_under_root(
624                 os.path.join(_proxies_dir(cfg), proxy_name), _proxies_dir(cfg)
625             )
626         except ValueError:
627             proxy_path = ""
628         if proxy_path and os.path.isfile(proxy_path):
629             return FileResponse(proxy_path, media_type="video/mp4")
630
631         # 2) Otherwise resolve + serve the source.
632         try:
633             input_ref = storage.resolve_input_ref(
634                 rec["filepath"], getattr(cfg, "storage", None)
635             )
636         except (
637             ValueError
638         ) as e: # unsupported scheme etc. ? a clean client error, never a 500
639             raise HTTPException(status_code=400, detail=str(e)) from e
640         except Exception as e: # noqa: BLE001 - a fetch miss is a bad-gateway, surfaced
(not swallowed)
641             raise HTTPException(
642                 status_code=502, detail=f"Could not resolve the source video: {e}"
643             ) from e
644
645         if input_ref.backend == "local":
646             local = input_ref.key
647         else:
648             try:
649                 local = _cached_remote_source(video_id, input_ref, cfg)
650             except HTTPException:
651                 raise
652             except Exception as e: # noqa: BLE001 - a fetch miss is surfaced (not
swallowed)
653                 raise HTTPException(
654                     status_code=502, detail=f"Could not resolve the source video: {e}"
655                 ) from e
656         if not os.path.isfile(local):
657             raise HTTPException(status_code=404, detail="Source video file not found.")
658
659         # 3) Warm a proxy in the background for next time.
660         threading.Thread(
661             target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
662         ).start()
663
664         media = "video/quicktime" if local.lower().endswith(".mov") else "video/mp4"
665         return FileResponse(local, media_type=media)
666
667
668     @app.post("/api/annotations")
669     def post_annotations(req: AnnotationsRequest, request: Request):
670         """Persist finished rally labels as the annotator CSV (P8), written verbatim to
671         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5

```

```

672     the video by content hash exactly as the training pipeline expects. Byte-compatible
with the VLC
673     plugin + browser extension. A later step (owner-gated) promotes labels into the
durable corpus.
674
675     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is
validated as a bare
676     filename so an untrusted value can't escape the labels dir (arbitrary write)."""
677     try:
678         edge_auth.resolve_tenant(request.headers, request.app.state.config)
679     except edge_auth.EdgeAuthError as e:
680         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
681
682     cfg = request.app.state.config
683     labels_dir = os.path.join(output_base(cfg), "labels")
684     try:
685         name = safe_output_filename(f"{req.video_id}.rallies.csv")
686     except ValueError as e:
687         raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
688     os.makedirs(labels_dir, exist_ok=True)
689     path = os.path.join(labels_dir, name)
690     try:
691         path = resolve_under_root(path, labels_dir)
692     except ValueError as e:
693         raise HTTPException(status_code=400, detail=str(e)) from e
694     with open(path, "w", encoding="utf-8", newline="") as f:
695         f.write(req.csv or "")
696     # Count CSV data rows (a line starting with an integer rally_number) for an honest
ack.
697     num_rows = sum(
698         1 for ln in (req.csv or "").splitlines() if ln[:1].isdigit() and "," in ln
699     )
700     logger.info(
701         "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
702     )
703     return {"status": "ok", "num_rows": num_rows, "csv_uri": f"labels/{name}"}
704
705
706     def _storage_settings(cfg) -> Dict[str, Any]:
707         """The storage config as a plain settings dict (NO secrets ? see StorageConfig)."""
708         sc = getattr(cfg, "storage", None)
709         if sc is None:
710             return {}
711         return sc.model_dump() if hasattr(sc, "model_dump") else dict(sc)
712
713
714     def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
715         """The durable, MD5-keyed destination inside the chosen medium:
716         ``<medium>/<video_id><ext>``. """
717         return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
718
719
720     def _medium_local_dir(medium_ref, storage_settings: Dict[str, Any]) -> Optional[str]:
721         """The LOCAL directory a ``put`` into this medium would write into (so a free-space
BLOCK can be
722         enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
723         ``local`` ? the folder; ``gdrive`` ? the resolved path under the Drive mount."""
724         if medium_ref.backend == "local":
725             return medium_ref.key or "."
726         if medium_ref.backend == "gdrive":
727             from backend.storage.capabilities import _gdrive_mount_root
728             root = _gdrive_mount_root(storage_settings)

```

```

729         if not root:
730             return None
731         rel = medium_ref.key.lstrip("/")
732         return os.path.join(root, *rel.split("/")) if rel else root
733     return None # s3 / gcs ? a bucket has no readable local capacity
734
735
736     @app.post("/api/assets", status_code=201)
737     def create_asset(req: AssetRequest, request: Request):
738         """Store a video into the user's chosen medium and register it as a library asset
739         (P4, D14).
740         SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root``
741         is set ? the
742         hosted alpha jails inputs and has no per-user medium credentials, so it stays
743         upload-to-local.
744         Edge-auth mirrors ``/api/process``. Exactly one source: an ``uploaded_path`` (a
745         local file from
746         the chunked upload) OR a ``source_uri`` (a video already at rest in a medium). An
747         uploaded file +
748         a REMOTE medium ? a durable, MD5-keyed copy is ``put`` into the medium while the
749         local upload
750         stays as the working copy; a ``local`` medium keeps the upload in place (it's
751         already on this box);
752         a pasted at-rest URI is registered as-is (no copy, D14b). The asset is keyed by the
753         whole-file
754         MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears
755         in
756         ``GET /api/videos``. """
757         try:
758             owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
759         except edge_auth.EdgeAuthError as e:
760             raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
761
762         cfg = request.app.state.config
763         sec = getattr(cfg, "security", None)
764         if getattr(sec, "allowed_video_root", None):
765             raise HTTPException(
766                 status_code=403,
767                 detail=(
768                     "storing into a custom medium is disabled on this hosted deployment "
769                     "(security.allowed_video_root is set); upload to local instead."
770                 ),
771             )
772         if bool(req.uploaded_path) == bool(req.source_uri):
773             raise HTTPException(
774                 status_code=400,
775                 detail="provide exactly one of 'uploaded_path' or 'source_uri'.",
776             )
777         if req.uploaded_path and not req.medium_uri:
778             raise HTTPException(
779                 status_code=400, detail="'medium_uri' is required when uploading a file."
780             )
781
782         storage_settings = _storage_settings(cfg)
783
784         # 1) Resolve a LOCAL working copy of the source + its whole-file MD5 (the join key,
785         D3).
786         if req.uploaded_path:
787             try:
788                 validate_input_path(req.uploaded_path, allowed_root=None)
789                 src_ref = storage.resolve_input_ref(req.uploaded_path, storage_settings)
790             except ValueError as e:
791                 raise HTTPException(status_code=400, detail=str(e)) from e
792             if src_ref.backend != "local" or not os.path.isfile(src_ref.key):

```

```

784         raise HTTPException(
785             status_code=404,
786             detail=f"uploaded file not found at '{req.uploaded_path}'",
787         )
788     local_working = src_ref.key
789 else:
790     try:
791         validate_input_path(req.source_uri, allowed_root=None)
792         src_ref = storage.resolve_input_ref(req.source_uri, storage_settings)
793     except ValueError as e:
794         raise HTTPException(status_code=400, detail=str(e)) from e
795     # A remote at-rest source is fetched to scratch to hash it (D3) ? reserve that
796     # scratch (P2).
797     if src_ref.backend != "local":
798         need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
799         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
800     try:
801         local_working = storage.acquire_local_input(
802             req.source_uri, getattr(cfg, "storage", None)
803         )
804     except ValueError as e:
805         raise HTTPException(status_code=400, detail=str(e)) from e
806     except Exception as e: # noqa: BLE001 ? a fetch miss is a 502, surfaced not
807         # swallowed
808         raise HTTPException(
809             status_code=502, detail=f"could not resolve source_uri: {e}"
810         ) from e
811     if not os.path.isfile(local_working):
812         raise HTTPException(
813             status_code=404, detail="source_uri did not resolve to a readable file."
814         )
815     cleanup_raw = req.source_uri or ""
816     try:
817         video_id = compute_video_id(local_working)
818         ext = os.path.splitext(local_working)[1] or ".mp4"
819
820         # 2) Decide the durable location + copy only when needed.
821         copied = False
822         if req.source_uri:
823             stored_uri = (
824                 req.source_uri
825                 # already at rest in its medium ? no copy (D14b)
826                 # medium_backend = src_ref.backend
827             )
828         else:
829             try:
830                 medium_ref = storage.parse_uri(req.medium_uri)
831             except ValueError as e:
832                 raise HTTPException(
833                     status_code=400, detail=f"invalid medium_uri: {e}"
834                 ) from e
835             medium_backend = medium_ref.backend
836             if medium_ref.backend == "local":
837                 # "This computer" ? the upload already lives here; it IS the durable
838                 # copy (no duplicate).
839                 stored_uri = os.path.abspath(local_working)
840             else:
841                 dest = _asset_dest_uri(req.medium_uri, video_id, ext)
842                 # Free-space: BLOCK a local/Drive-mount destination (D11); a cloud
843                 # bucket is warn-only.
844                 local_dir = _medium_local_dir(medium_ref, storage_settings)
845                 if local_dir is not None:
846                     require_free_bytes(
847                         local_dir, os.path.getsize(local_working), stage="store"
848                     )

```



```

845         else:
846             logger.info(
847                 "[ASSETS] cloud medium %s ? capacity not blocked (cost/quota
warn only).",
848                 medium_ref.backend,
849             )
850         try:

```

10. user (2026-07-04T11:49:19.962Z)

```

850         try:
851             stored_uri = storage.put(
852                 local_working, dest, config=storage_settings
853             )
854             copied = True
855         except ValueError as e:
856             raise HTTPException(status_code=400, detail=str(e)) from e
857         except Exception as e: # noqa: BLE001 ? a put failure is surfaced,
never silently dropped
858             logger.error(
859                 "[ASSETS] put failed video_id=%s dest=%s: %s",
860                 video_id,
861                 dest,
862                 e,
863                 exc_info=True,
864             )
865             raise HTTPException(
866                 status_code=502, detail=f"could not store into the medium: {e}"
867             ) from e
868
869         # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my
videos").
870         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
871             raise HTTPException(status_code=500, detail="failed to register the asset.")
872         logger.info(
873             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
874             video_id,
875             medium_backend,
876             stored_uri,
877             copied,
878             owner_id,
879         )
880         return JSONResponse(
881             status_code=201,
882             content={
883                 "video_id": video_id,
884                 "medium": medium_backend,
885                 "stored_uri": stored_uri,
886                 "copied": copied,
887                 "sport": req.sport,
888                 "status": "stored",
889             },
890         )
891         finally:
892             if cleanup_raw:
893                 storage.cleanup_acquired_local_input(
894                     cleanup_raw, local_working, getattr(cfg, "storage", None)
895                 )
896
897
898         @app.post("/api/process", status_code=202)
899         def process_video(req: ProcessRequest, request: Request):
900             """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
901
902             Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the

```

```

903     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
904     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
905     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
906     ""
907     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
single-user,
908     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
909     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
910     try:
911         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
912     except edge_auth.EdgeAuthError as e:
913         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
914
915     allowed_root = getattr(
916         getattr(request.app.state.config, "security", None), "allowed_video_root", None
917     )
918     # In hosted mode a bucket URI is allowed only when it's under the configured
input_root (the
919     # designated bucket) ? not an arbitrary bucket the runtime SA happens to reach
(tenant isolation).
920     allowed_uri_root = getattr(
921         getattr(request.app.state.config, "storage", None), "input_root", None
922     )
923     try:
924         validate_input_path(
925             req.video_path, allowed_root=allowed_root, allowed_uri_root=allowed_uri_root
926         )
927     # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
928     # scheme, which we map to a clean 400 (an unknown scheme must never 500).
929     input_ref = storage.resolve_input_ref(
930         req.video_path, getattr(request.app.state.config, "storage", None)
931     )
932     except ValueError as e:
933         raise HTTPException(status_code=400, detail=str(e)) from e
934     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
935     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
936     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
937     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
938     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
939     # rejects a non-local URI as out-of-root.
940     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
941         raise HTTPException(
942             status_code=404, detail=f"Video file not found at '{req.video_path}'"
943         )
944     # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail
fast with 507
945     # if this box can't hold that copy. Best-effort ? an unstatable remote
(network/SDK) or an
946     # unreadable temp fs skips the guard (measuring must not deny a legit job); the
fetch itself
947     # still fails loudly downstream if space genuinely runs out. Local inputs need no
copy ? skipped.
948     if input_ref.backend != "local":
949         need = _remote_input_size_bytes(
950             input_ref, getattr(request.app.state.config, "storage", None)
951         )
952         require_free_bytes(tempfile.gettempdir(), need, stage="fetch")

```

```

953         if req.segmenter and not segmenter_registry.get(req.segmenter):
954             available = list(segmenter_registry.list_available().keys())
955             raise HTTPException(
956                 status_code=400,
957                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}",
958             )
959
960         job_id = uuid.uuid4().hex
961         if not request.app.state.db.create_job(
962             job_id,
963             req.video_path,
964             req.segmenter,
965             req.player_pool,
966             req.max_frames,
967             owner_id=owner_id,
968             locale=req.locale,
969             compute=req.compute,
970             force=req.force,
971         ):
972             raise HTTPException(status_code=500, detail="Failed to persist job record.")
973         _get_executor().submit(
974             _drain_due_jobs, request.app.state.config, request.app.state.db
975         )
976         return JSONResponse(
977             status_code=202,
978             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
979         )
980
981
982     @app.get("/api/jobs/{job_id}/cost")
983     def get_job_cost(job_id: str, request: Request):
984         """Per-job cost (COMPUTE_DECOUPLED_SERVING M8, D8). ``cost_usd`` is the source of
truth (matches
985         GCP billing); ``cost_inr`` is a display conversion at the configured FX rate.
``cost_usd`` is
986         null until billing records it (default-OFF / the placeholder pricebook ? 0)."""
987         cost = request.app.state.db.get_job_cost(job_id)
988         if cost is None:
989             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
990         fx = request.app.state.config.billing.fx_inr_per_usd
991         usd = cost.get("cost_usd")
992         cost["cost_inr"] = billing.to_inr(usd, fx) if usd is not None else None
993         cost["fx_inr_per_usd"] = fx
994         return cost
995
996
997     @app.get("/api/videos/{video_id}/cost")
998     def get_video_cost(video_id: str, request: Request):
999         """Aggregate per-video cost across its jobs (M8): ``total_cost_usd`` +
``total_cost_inr`` + the
1000         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
several."""
1001         agg = request.app.state.db.get_video_cost(video_id)
1002         fx = request.app.state.config.billing.fx_inr_per_usd
1003         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
1004         agg["fx_inr_per_usd"] = fx
1005         return agg
1006
1007
1008     @app.get("/api/jobs/{job_id}")
1009     def get_job(job_id: str, request: Request):
1010         """Job state: {status, progress, result, reports}. ``status`` = execution state
1011         (queued|running|done|failed); the pipeline verdict is result["status"]."""
1012         job = request.app.state.db.get_job(job_id)

```

```

1013         if not job:
1014             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
1015         result = job.get("result")
1016         return {
1017             "job_id": job["id"],
1018             "status": job["status"],
1019             "progress": job.get("progress"),
1020             "video_id": job.get("video_id"),
1021             "error": job.get("error"),
1022             "result": result,
1023             "reports": (result or {}).get("reports"),
1024             "created_at": job.get("created_at"),
1025             "started_at": job.get("started_at"),
1026             "finished_at": job.get("finished_at"),
1027         }
1028
1029
1030 # --- Chunked upload (B2, PR-2) -----
1031 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
1032 # in
1033 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
1034 # via
1035 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
1036 # sequential-part logic, and abort cleanup ? is unchanged.
1037 from backend.api import uploads as uploads_api
1038
1039 app.include_router(uploads_api.router)
1040
1041 # --- Highlight download (B3, PR-2) -----
1042
1043 @app.get("/api/highlights/{video_id}/{filename}")
1044 def download_highlight(video_id: str, filename: str, request: Request):
1045     """Stream a compiled highlight reel ? `filename` is the bare name given to
1046     /api/compile
1047     (which only writes into output_dir; the browser previously got back an unreachable
1048     server-local path)."""
1049     try:
1050         name = safe_output_filename(filename)
1051         except ValueError as e:
1052             raise HTTPException(status_code=400, detail=str(e)) from e
1053         if not request.app.state.db.get_video(video_id):
1054             raise HTTPException(status_code=404, detail="Indexed video record not found.")
1055         # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
1056         URL (bytes flow
1057         # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
1058         /tmp). The
1059         # exists/sign logic (with the active storage config) lives in
1060         orchestration.signed_reel_url to keep
1061         # this module lean; best-effort ? any error falls back to the local file below.
1062         try:
1063             signed = signed_reel_url(request.app.state.config, video_id, name)
1064             if signed:
1065                 return RedirectResponse(signed, status_code=307)
1066         except Exception as e: # noqa: BLE001 ? signing/exists hiccup ? serve the local
1067         file instead
1068             logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
1069                             video_id, e)
1070         output_dir = (
1071             getattr(
1072                 getattr(request.app.state.config, "output", None), "output_dir", "output"
1073             )
1074             or "output"
1075         )

```

```

1070     path = os.path.join(output_dir, name)
1071     try:
1072         path = resolve_under_root(path, output_dir)
1073     except ValueError as e:
1074         raise HTTPException(status_code=400, detail=str(e)) from e
1075     if not os.path.isfile(path):
1076         raise HTTPException(
1077             status_code=404,
1078             detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1079         )
1080     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1081     return FileResponse(path, media_type=media, filename=name)
1082
1083
1084 @app.post("/api/query")
1085 def query_play_segments(req: QueryRequest, request: Request):
1086     filters = {
1087         "server": req.server,
1088         "receiver": req.receiver,
1089         "duration_min": req.duration_min,
1090         "event_type": req.event_type,
1091     }
1092     segments = request.app.state.db.query_play_segments(req.video_id, filters)
1093     return {"play_segments": segments}
1094
1095
1096 @app.post("/api/compile", status_code=202)
1097 def compile_highlights(req: CompileRequest, request: Request):
1098     """Submit a reel-COMPILE job (202 + job_id); poll GET /api/jobs/{job_id} for the
1099     reel (#329).
1100     Async, mirroring /api/process: the ffmpeg stitch (trim + re-encode each clip +
1101     concat + watermark)
1102     can run minutes on CPU ? past the ~100 s edge/proxy timeout ? so doing it
1103     synchronously returned a
1104     Cloudflare 524 while the engine kept going (the reel built, but the client saw
1105     "failed" and never
1106     got the URL). The cheap checks (video exists, segments match + survive the floor,
1107     safe filename)
1108     fast-fail HERE with a 4xx; the stitch runs on the worker. On `done`, the job's
1109     `result` carries
1110     {filepath, download_url} ? the same shape the sync endpoint returned, now under
1111     /api/jobs/{id}."""
1112     # M9 edge-auth tenant (DEFAULT-OFF ? None); a missing/spoofed identity fails 401
1113     before queuing.
1114     try:
1115         owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
1116     except edge_auth.EdgeAuthError as e:
1117         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
1118     try:
1119         video, _kept, out_name = _resolve_compile(
1120             request.app.state.db,
1121             request.app.state.config,
1122             req.video_id,
1123             req.segment_ids,
1124             req.output_filename,
1125         )
1126     except _CompileError as e:
1127         raise HTTPException(status_code=e.status_code, detail=e.detail) from e
1128
1129     job_id = uuid.uuid4().hex
1130     # Persist the video's SOURCE ref (the NOT NULL video_path); the worker re-fetches
1131     the row by id so
1132     # the freshest filepath wins. owner_id/locale ride along for M9 multi-tenant + the
1133     localized reel.

```

```

1125     if not request.app.state.db.create_compile_job(
1126         job_id,
1127         req.video_id,
1128         video["filepath"],
1129         req.segment_ids,
1130         out_name,
1131         locale=req.locale,
1132         owner_id=owner_id,
1133     ):
1134         raise HTTPException(
1135             status_code=500, detail="Failed to persist compile job record."
1136         )
1137     _get_executor().submit(
1138         _drain_due_jobs, request.app.state.config, request.app.state.db
1139     )
1140     return JSONResponse(
1141         status_code=202,
1142         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
1143     )
1144
1145
1146 # --- CLI Debug Utility & Orchestration ---
1147 def run_cli_diagnose_clip(video_path: str, timestamp: float):
1148     """CLI debugging utility to examine segment properties around a timestamp."""
1149     print("=" * 60)
1150     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
1151     print("=" * 60)
1152
1153     cfg = load_config() # legacy wrapper still works, returns dict for now
1154
1155     # 1. Validation check diagnostics
1156     print("[DIAGNOSTIC] Running validation framework...")
1157     results = registry.run_all(video_path, cfg)
1158     for r in results:
1159         status_symbol = "[PASS]" if r.passed else "[FAIL]"
1160         print(f"  {status_symbol} {r.validator_name}: {r.message}")
1161         if r.details:
1162             print(f"    Details: {r.details}")
1163
1164     # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
1165     print("-" * 60)
1166     print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
1167     # Lazy import: cv2 is NOT installed in the LIGHT tier (torch-free default)
1168     # and this --debug-clip path is CLI-only, not on the served API path.
1169     import cv2
1170
1171     cap = cv2.VideoCapture(video_path)
1172     if not cap.isOpened():
1173         print("[FAIL] OpenCV could not open video.")
1174         return
1175
1176     fps = cap.get(cv2.CAP_PROP_FPS)
1177     total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
1178
1179     start_frame = max(0, int((timestamp - 3.0) * fps))
1180     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
1181
1182     # Measure frame-by-frame changes in sample region
1183     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
1184     prev_gray = None
1185     motions = []
1186
1187     for _ in range(start_frame, end_frame):
1188         ret, frame = cap.read()
1189         if not ret:

```

```

1190         break
1191         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
1192         gray = cv2.GaussianBlur(gray, (21, 21), 0)
1193
1194         if prev_gray is not None:
1195             diff = cv2.absdiff(prev_gray, gray)
1196             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
1197             motion_ratio = cv2.countNonZero(thresh) / (
1198                 thresh.shape[0] * thresh.shape[1]
1199             )
1200             motions.append(motion_ratio)
1201             prev_gray = gray
1202
1203     cap.release()
1204
1205     if motions:
1206         avg_motion = sum(motions) / len(motions)
1207         # Support both legacy dict (from wrapper) and future direct model
1208         if hasattr(cfg, "indexing"):
1209             threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
1210         else:
1211             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
1212         print(f" Sample Frame Count: {len(motions)}")
1213         print(
1214             f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})"
1215         )
1216         if avg_motion > threshold:
1217             print(
1218                 " Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play."
1219             )
1220         else:
1221             print(
1222                 " Result: [DEAD TIME] Idle movement. Segment classified as dead time."
1223             )
1224     else:
1225         print(" [ERROR] No visual frames were extracted in the sample range.")
1226
1227     print("=" * 60)
1228
1229
1230     def _enforce_startup_or_exit(config) -> None:
1231         """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL
finding BEFORE
1232         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
storage) OR, for
1233         the exposed server (``include_exposure=True``, M9 hardening), a half-configured
exposure (edge
1234         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
no
1235         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
probe here ?
1236         the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted so the
1237         fail-fast wiring + the default-OFF no-op are unit-testable."""
1238         try:
1239             enforce_startup_checks(config, logger=logger, include_exposure=True)
1240         except StartupCheckError as e:
1241             logger.error("[STARTUP] refusing to start: %s", e)
1242             sys.exit(1)
1243
1244
1245     def _maybe_sweep_gemini_orphans(config) -> None:

```

```

1246         """01/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
crashed run.
1247         A no-op unless the configured provider is Gemini AND its API key is present. Never
raises ?
1248         a failed sweep is logged and swallowed. The Gemini client is built with an HTTP
timeout so
1249         a hung connection can't stall startup indefinitely. Extracted for unit tests."""
1250     try:
1251         provider_name = getattr(config, "ai_provider", "")
1252         if provider_name != "gemini":
1253             return
1254         api_key = os.environ.get("GEMINI_API_KEY")
1255         if not api_key:
1256             return
1257         provider = provider_registry.get(
1258             "gemini", api_key=api_key, model_name=config.ai_model
1259         )
1260         deleted = provider.cleanup_orphaned_files()
1261         if deleted:
1262             logger.info(
1263                 "[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).", deleted
1264             )
1265     except Exception as e:
1266         logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1267
1268
1269 def main():
1270     parser = argparse.ArgumentParser(
1271         description="Sports Highlight Indexer Orchestrator"
1272     )
1273     parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1274     parser.add_argument(
1275         "--output-dir",
1276         default=None,
1277         help="Directory where processed clips/highlights are saved (overrides
config.json output.output_dir)",
1278     )
1279     parser.add_argument(
1280         "--setup",
1281         action="store_true",
1282         help="Launch dependency checker and setup utility",
1283     )
1284     parser.add_argument(
1285         "--debug-clip",
1286         type=float,
1287         help="Timestamp (in seconds) of a video clip to run detailed diagnostics on",
1288     )
1289     parser.add_argument(
1290         "--server", action="store_true", help="Run the FastAPI local API server"
1291     )
1292     parser.add_argument(
1293         "--app",
1294         action="store_true",
1295         help="Launch the app: start the local server (localhost-only) and open the "
1296         "bundled UI in your browser. Falls back to a free port if --port is busy.",
1297     )
1298     parser.add_argument(
1299         "--port", type=int, default=8000, help="Port to run the FastAPI server on"
1300     )
1301     parser.add_argument(
1302         "--host",
1303         default=None,
1304         help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0 when "
1305         "security.edge_auth_enabled. Expose on a network ONLY behind an auth gate.",
1306     )

```



```

1307     parser.add_argument(
1308         "--max-frames",
1309         type=int,
1310         help="Limit processing to the first N sub-sampled frames for debugging",
1311     )
1312
1313     available_segmenters = list(segmenter_registry.list_available().keys())
1314     parser.add_argument(
1315         "--segmenter",
1316         choices=available_segmenters,
1317         help=f"Choose rally detection segmenter. Available: {available_segmenters}",
1318     )
1319     parser.add_argument(
1320         "--trim-start",
1321         type=float,
1322         help="Start time in seconds for the debug trim segment feature",
1323     )
1324     parser.add_argument(
1325         "--trim-end",
1326         type=float,
1327         help="End time in seconds for the debug trim segment feature",
1328     )
1329     parser.add_argument(
1330         "--trim-output",
1331         help="Output filename for the debug trimmed segment (saves in output-dir unless
absolute path)",
1332     )
1333
1334     args = parser.parse_args()
1335
1336     if args.setup:
1337         # Invoke the diagnostics/bootstrap script (renamed from the old root
1338         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
1339         # build system). Resolve its path from this file so `indexer --setup`
1340         # works regardless of the caller's cwd.
1341         import subprocess
1342
1343         doctor_path = os.path.join(
1344             os.path.dirname(os.path.abspath(__file__)),
1345             "scripts",
1346             "doctor.py",
1347         )
1348         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
1349         subprocess.run([sys.executable, doctor_path])
1350         return
1351
1352     if args.trim_start is not None and args.trim_end is not None:
1353         if not args.video_path:
1354             print("[ERROR] Please provide --video-path to extract a segment.")
1355             return
1356
1357         # Determine output path
1358         out_filename = args.trim_output or "trimmed_segment.mp4"
1359         if os.path.isabs(out_filename):
1360             out_path = out_filename
1361             out_dir = os.path.dirname(out_path)
1362             out_name = os.path.basename(out_path)
1363         else:
1364             out_dir = resolve_output_dir(
1365                 args.output_dir or "output"
1366             ) # P0a: app-home aware (CWD-identical when unset)
1367             out_path = os.path.join(out_dir, out_filename)
1368             out_name = out_filename
1369
1370         os.makedirs(out_dir, exist_ok=True)

```

```

1371         print(
1372             f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}..."
1373         )
1374
1375         # global_config isn't initialized this early; load the typed config so the trim
1376         # clip carries the same configured stitching paddings + watermark as a real
1377         # highlight (watermark default-on).
1378         stitching = load_app_config().stitching
1379         compiler = VideoCompiler(
1380             out_dir,
1381             begin_padding=stitching.begin_padding,
1382             end_padding=stitching.end_padding,
1383             watermark=stitching.watermark,
1384         )
1385         try:
1386             res_path = compiler.compile_highlights(
1387                 args.video_path, [(args.trim_start, args.trim_end)], out_name
1388             )
1389             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
1390         except Exception as e:
1391             print(f"[ERROR] Failed to compile trimmed segment: {e}")
1392         return
1393
1394     if args.debug_clip is not None:
1395         if not args.video_path:
1396             print("[ERROR] Please provide --video-path to debug a specific clip.")
1397             return
1398         run_cli_diagnose_clip(args.video_path, args.debug_clip)
1399         return
1400
1401     # P2b (PACKAGING_PLAN.md gate 08): batteries-included first run for the friendly
1402     `--app`
1403     # launcher. Seed a truly-local, torch-free LIGHT default config (detector 'motion' ?
offline,
1404     # no API key, no GPU, no cloud bills) at the resolved app-home
(default_config_path(), which
1405     # roots under RALLY_APP_HOME when the packaged shortcut pins it) BEFORE
load_app_config reads
1406     # it ? so a fresh `pip`/`uv tool install` + `indexer --app` just works. NEVER
overwrites an
1407     # existing config (a user's edits / the P2c wizard's choices are preserved). Scoped
to --app
1408     # (the packaged app mode); `--server` and the CLI single-shot are byte-identical (no
seeding).
1409     if args.app:
1410         seeded_path, seeded = write_light_default_config()
1411         if seeded:
1412             print(
1413                 f"[APP] First run: seeded a truly-local default config at {seeded_path}
"
1414                 f"(detector 'motion' ? offline, no API key or GPU needed). "
1415                 f"Use the in-app setup to switch to Gemini."
1416             )
1417
1418     # Initialize Global Config and DB
1419     global global_config, db_instance
1420     global_config = load_app_config() # returns typed AppConfig
1421     _enforce_startup_or_exit(
1422         global_config
1423     ) # M5: fail fast on an incoherent ACTIVE profile
1424
1425     # The CLI --output-dir overrides the configured output directory ONLY when
explicitly
1426     # passed. When not passed, the config.json output.output_dir value takes effect.

```

```

1426      # It now lives on the typed model (OutputConfig.output_dir), so every consumer ?
including
1427      # /api/compile ? reads the same value instead of a write-only runtime hack.
1428      # P0a: a RELATIVE output dir roots under the app-home (RALLY_APP_HOME); with the env
1429      # unset, resolve_output_dir("output") == "output" (CWD-relative) ? byte-identical to
before.
1430      if args.output_dir is not None:
1431          global_config.output.output_dir = resolve_output_dir(args.output_dir)
1432      else:
1433          global_config.output.output_dir = resolve_output_dir(
1434              global_config.output.output_dir
1435          )
1436      os.makedirs(global_config.output.output_dir, exist_ok=True)
1437
1438      db_path = os.path.join(
1439          global_config.output.output_dir, global_config.output.db_name
1440      )
1441
1442      # P0e: refuse a SECOND instance against the same DB BEFORE the destructive stale-job
sweep below.
1443      # The job worker is in-process, so a concurrent process opening this DB would sweep
THIS one's
1444      # in-flight jobs to 'failed'. The OS lock (held for the process lifetime; auto-
released on exit
1445      # or crash) is keyed to db_path, so a different --output-dir/DB still runs
concurrently.
1446      global _instance_lock
1447      try:
1448          _instance_lock = acquire_lock(db_path + ".lock")
1449      except OSError as exc:
1450          # The lock path isn't writable (read-only dir / perms). Don't crash and don't
falsely claim a
1451          # second instance ? warn and proceed WITHOUT the guard (a lock-infra failure
must not block a
1452          # legitimate single instance).
1453          logger.warning(
1454              "[JOBS] Could not create the single-instance lock at %s.lock (%s);
proceeding "
1455              "WITHOUT the second-instance guard.",
1456              db_path,
1457              exc,
1458          )
1459          _instance_lock = None
1460      else:
1461          if _instance_lock is None:
1462              print(
1463                  f"[FATAL] Another instance is already using this database ({db_path}).
Refusing to "
1464                  f"start ? a second instance would corrupt in-flight job state. Stop the
other one, or "
1465                  f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance."
1466              )
1467              sys.exit(1)
1468
1469      db_instance = Database(db_path)
1470
1471      # Crash recovery (#16, #329): jobs left queued/running by a previous process are
dead ?
1472      # the executor is in-process. Compile jobs (idempotent CPU stitch) are RE-QUEUED so
a reel
1473      # interrupted by a Cloud Run scale-down finishes on the next drain instead of
failing the
1474      # client; process jobs (GPU, non-idempotent) are failed so pollers see a terminal
state.

```

```

1475         requeued, failed = db_instance.recover_stale_jobs()
1476     if requeued or failed:
1477         logger.warning(
1478             f"[JOBS] Recovered stale jobs from a previous run ? {requeued} compile re-
queued, "
1479             f"{failed} process failed."
1480         )
1481
1482     # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1483     # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1484     # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1485     _maybe_sweep_gemini_orphans(global_config)
1486
1487     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1488     if args.video_path and not args.server and not args.app:
1489         print(f"[CLI] Processing video file: {args.video_path}")
1490         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1491         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1492         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1493         storage_cfg = getattr(global_config, "storage", None)
1494         try:
1495             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1496         except ValueError as e:
1497             print(f"[FAIL] {e}")
1498             return
1499         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1500             print(f"[FAIL] Video file not found: {args.video_path}")
1501             return
1502         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1503         try:
1504             video_id = compute_video_id(local_video)
1505             was_reingest = db_instance.get_video(video_id) is not None
1506
1507             # Validate (with-context variant surfaces ffprobe_meta for the report)
1508             print("[CLI] Validating...")
1509             val_results, val_context = registry.run_all_with_context(
1510                 local_video, global_config
1511             )
1512             failures = [r for r in val_results if not r.passed]
1513
1514             # Save video status
1515             status = "failed" if failures else "processed"
1516             db_instance.add_video(
1517                 video_id, args.video_path, status, [r.model_dump() for r in val_results]
1518             )
1519
1520             seg_name = args.segmenter or getattr(
1521                 getattr(global_config, "indexing", None), "default_segmenter", "motion"
1522             )
1523             segmenter_actual = None
1524             segmentation_ran = False
1525             try:
1526                 print("\n" + "=" * 40)
1527                 print("VALIDATION SUMMARY")
1528                 print("=" * 40)
1529                 for r in val_results:
1530                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
1531                     print(f"{status_symbol} {r.validator_name}: {r.message}")

```

```

1532         print("=" * 40 + "\n")
1533
1534         if failures:
1535             print("[FAIL] Video failed checks. Indexing halted.")
1536             return
1537
1538         # Index
1539         segmenter_cls = segmenter_registry.get(seg_name)
1540         if not segmenter_cls:
1541             print(f"[FAIL] Segmenter '{seg_name}' not found.")
1542             return
1543
1544         print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
1545         segmenter_actual = seg_name
1546         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
1547         segmenter = segmenter_cls(db_instance, global_config)
1548         result = segmenter.process_video(
1549             video_id, local_video, max_frames=args.max_frames
1550         )
1551         segmentation_ran = True
1552
1553         if isinstance(result, Failure):
1554             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
1555             print("[FAIL] Indexing halted for video.")
1556             segments = []
1557         else:
1558             segments = result.value if hasattr(result, "value") else result
1559             print(
1560                 f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}"
1561             )
1562             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
1563             _finalize_reingest(db_instance, video_id, segments, play_wm, cand_wm)
1564         finally:
1565             # Always emit the per-video observability report. Never raises.
1566             paths = emit_indexer_report(
1567                 db=db_instance,
1568                 video_id=video_id,
1569                 # Resolved LOCAL path, not the raw URI (see #397) ? a remote URI
would write a
1570                 # literal scheme dir (gcs:/, gs:/, local:/) at the repo root. Source
URI is kept
1571                 # in DB provenance via db.add_video.
1572                 video_path=local_video,
1573                 config=_report_config_for_input(global_config, args.video_path),
1574                 val_results=val_results,
1575                 val_context=val_context,
1576                 segmenter_requested=args.segmenter,
1577                 segmenter_actual=segmenter_actual,
1578                 was_reingest=was_reingest,
1579                 segmentation_ran=segmentation_ran,
1580             )
1581             if paths:
1582                 print(f"[REPORT] Technical report : {paths.get('technical_md')}")
1583                 print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
1584                 print(f"[REPORT] Machine JSON      : {paths.get('json')}")
1585             finally:
1586                 storage.cleanup_acquired_local_input(
1587                     args.video_path, local_video, storage_cfg
1588                 )
1589             return
1590
1591     # Start Local Server Mode
1592     if args.app or args.server or not args.video_path:

```

```

1593         if args.app:
1594             # P2: the friendly launcher ? always localhost-only (use --server for
network exposure),
1595             # auto-pick a free port if --port is busy, and open the bundled UI once the
server is up.
1596             bind_host = "127.0.0.1"
1597             port = _resolve_app_port(bind_host, args.port)
1598             if port != args.port:
1599                 print(f"[APP] Port {args.port} is busy ? using {port} instead.")
1600             url = f"http://{bind_host}:{port}/"
1601             # The browser-open health poll sends Host: 127.0.0.1 ? it relies on loopback
staying in
1602             # the Host allowlist (the default; don't set RALLY_ALLOWED_HOSTS to a non-
loopback-only set
1603             # with --app, or the poll 400s and the browser just won't auto-open ? the
server still runs).
1604             print(
1605                 f"[APP] Starting KhelSutra ? opening {url} in your browser "
1606                 f"(set RALLY_NO_BROWSER=1 to skip). Press Ctrl+C to stop."
1607             )
1608             threading.Thread(
1609                 target=_open_browser_when_ready,
1610                 args=(url, url + "api/health"),
1611                 daemon=True,
1612             ).start()
1613         else:
1614             bind_host = _resolve_bind_host(
1615                 global_config, args.host
1616             ) # P0c: config/profile-driven (default 127.0.0.1)
1617             port = args.port
1618             print(
1619                 f"[SERVER] Starting local FastAPI server on
http://{bind_host}:{port}..."
1620             )
1621             if bind_host not in ("127.0.0.1", "localhost", "::1"):
1622                 print(
1623                     f"[SERVER] ? Binding {bind_host} exposes the API on the network ?
ensure an auth "
1624                     f"gate (edge_auth_enabled / a reverse proxy) is in front, and set "
1625                     f"RALLY_ALLOWED_HOSTS to your public host(s) (the Host-header guard
defaults to "
1626                     f"loopback-only and will 400 forwarded requests otherwise)."
1627                 )
1628             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
1629             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
1630             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
1631             configure_drain(global_config) # drain concurrency from config before the
executor exists (#466)
1632             _get_executor().submit(_drain_due_jobs, global_config, db_instance)
1633             # Stamp shared state on the app so route handlers access it via
request.app.state
1634             # instead of the Optional module-level globals (CODE_CLEANUP_AUDIT P5 / D2).
1635             create_app(global_config, db_instance)
1636             uvicorn.run(app, host=bind_host, port=port)
1637
1638
1639     if __name__ == "__main__":
1640         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
resolve the
1641         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
a SEPARATE
1642         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
1643         # `backend.main.db_instance` = None; the async-job drain then crashes silently on

```

```

1644 # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
through
1645 # `backend.main` so `main()` configures the module everyone actually reads. (No
recursion:
1646 # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
fire.)
1647 import backend.main as _bm
1648
1649 _bm.main()
1650

```

11. user (2026-07-04T11:49:27.601Z)

```

1 import json
2 import logging
3 from typing import Any, Dict, List, Optional, Tuple
4
5 from backend import billing # pure cost math (no backend imports -> no cycle)
6
7 logger = logging.getLogger(__name__)
8
9
10 class DatabaseJobsMixin:
11     def _connect(self): # pragma: no cover - implemented by Database facade
12         raise NotImplementedError
13
14     def _execute_write( # pragma: no cover - implemented by Database facade
15         self, sql: str, params: Tuple[Any, ...], error_msg: str
16     ) -> bool:
17         raise NotImplementedError
18
19     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
20     # Writers follow the repo convention (swallow + logger.error, return bool); the
21     # worker logs loudly on a failed transition so a job can't silently wedge.
22
23     def create_job(
24         self,
25         job_id: str,
26         video_path: str,
27         segmenter: Optional[str] = None,
28         player_pool: Optional[List[str]] = None,
29         max_frames: Optional[int] = None,
30         policy: Optional[Dict[str, Any]] = None,
31         owner_id: Optional[str] = None,
32         locale: Optional[str] = None,
33         compute: Optional[str] = None,
34         force: bool = False,
35     ) -> bool:
36         """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
37         (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy.
``owner_id``
38         (M9, D9) is the verified tenant from edge auth ? NULL for the local single-user
(default).
39         ``locale`` (v7) is the UI language the submission came from ? NULL when
unrecorded.
40         ``compute`` (v8) is the SPA compute-picker choice ("local"/"cloud") the worker
reconstructs +
41         honours; NULL ? the server default (byte-identical to pre-picker behaviour).
``force`` opts
42         into deliberate reprocessing; false leaves params NULL for old-row
compatibility."""
43         params = json.dumps({"force": True}) if force else None
44         return self._execute_write(
45             "INSERT INTO jobs (id, video_path, segmenter, player_pool, max_frames,

```

```

status, policy, "
46         "owner_id, locale, compute, params) VALUES (?, ?, ?, ?, ?, 'queued', ?, ?,
?, ?, ?)",
47     (
48         job_id,
49         video_path,
50         segmenter,
51         json.dumps(player_pool) if player_pool is not None else None,
52         max_frames,
53         json.dumps(policy) if policy is not None else None,
54         owner_id,
55         locale,
56         compute,
57         params,
58     ),
59     f"Failed to create job {job_id}",
60 )
61
62 def create_compile_job(
63     self,
64     job_id: str,
65     video_id: str,
66     video_path: str,
67     segment_ids: List[int],
68     output_filename: str,
69     locale: Optional[str] = None,
70     owner_id: Optional[str] = None,
71 ) -> bool:
72     """Insert an async REEL-COMPILE job (#329) in state 'queued', `kind`='compile'.
The stitch
73     params (segment_ids / output_filename / locale) ride the JSON `params` column
since they don't
74     map onto the process columns; `video_path` is the target video's source ref (the
table's
75     NOT NULL column) and `video_id` is set up-front (so /api/jobs surfaces it
immediately). The
76     worker dispatches on `kind` and runs the ffmpeg stitch off the request
thread."""
77     params = json.dumps(
78         {
79             "segment_ids": list(segment_ids),
80             "output_filename": output_filename,
81             "locale": locale,
82         }
83     )
84     return self._execute_write(
85         "INSERT INTO jobs (id, video_path, video_id, status, owner_id, locale, kind,
params) "
86         "VALUES (?, ?, ?, 'queued', ?, ?, 'compile', ?)",
87         (job_id, video_path, video_id, owner_id, locale, params),
88         f"Failed to create compile job {job_id}",
89     )
90
91 def count_jobs_by_locale(self) -> Dict[str, int]:
92     """PMF analytics: how many jobs were submitted per UI locale. Unrecorded locales
(NULL ?
93     CLI / pre-v7 / older clients) bucket under '(unset)'. Returns {locale: count},
busiest first."""
94     with self._connect() as conn:
95         rows = (
96             conn.cursor()
97             .execute(
98                 "SELECT COALESCE(locale, '(unset)') AS loc, COUNT(*) AS n "
99                 "FROM jobs GROUP BY loc ORDER BY n DESC, loc"
100             )

```



```

101         .fetchall()
102     )
103     return {str(r[0]): int(r[1]) for r in rows}
104
105 def mark_job_running(self, job_id: str) -> bool:
106     return self._execute_write(
107         "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
108         "progress='starting' WHERE id = ?",
109         (job_id,),
110         f"Failed to mark job {job_id} running",
111     )
112
113 def set_job_progress(self, job_id: str, progress: str) -> bool:
114     return self._execute_write(
115         "UPDATE jobs SET progress=? WHERE id = ?",
116         (progress, job_id),
117         f"Failed to set progress for job {job_id}",
118     )
119
120 def set_job_video_id(self, job_id: str, video_id: str) -> bool:
121     return self._execute_write(
122         "UPDATE jobs SET video_id=? WHERE id = ?",
123         (video_id, job_id),
124         f"Failed to set video_id for job {job_id}",
125     )
126
127 def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
128     """Job EXECUTED to completion. The pipeline's own verdict (success/failed
129     validation) is inside `result` ? job status 'done' means 'the worker
130     finished'."""
131     return self._execute_write(
132         "UPDATE jobs SET status='done', progress='finished', result=?, "
133         "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
134         (json.dumps(result), job_id),
135         f"Failed to mark job {job_id} done",
136     )
137
138 def mark_job_failed(self, job_id: str, error: str) -> bool:
139     """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
140     return self._execute_write(
141         "UPDATE jobs SET status='failed', error=?, "
142         "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
143         (error, job_id),
144         f"Failed to mark job {job_id} failed",
145     )
146
147 def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
148     """Fetch one job; `result`/`player_pool` JSON deserialized."""
149     with self._connect() as conn:
150         row = (
151             conn.cursor()
152             .execute("SELECT * FROM jobs WHERE id = ?", (job_id,))
153             .fetchone()
154         )
155         if not row:
156             return None
157         d = dict(row)
158         d["result"] = json.loads(d["result"]) if d["result"] else None
159         d["player_pool"] = (
160             json.loads(d["player_pool"]) if d["player_pool"] else None
161         )
162         d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
163         return d
164
165 # --- Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8)

```

```

-----
165     def get_pricebook_rates(
166         self, version: str, gpu_type: Optional[str] = None
167     ) -> Dict[str, float]:
168         """Resolve a flat ``{resource: unit_price_usd}`` for a pricebook ``version``.
The per-second
169         GPU rate is selected by ``gpu_type``; non-GPU resources (wall/egress) carry
``gpu_type=None``.
170         An unknown version ? ``{}`` (so every cost computes to 0.0)."""
171         gt = gpu_type or ""
172         rates: Dict[str, float] = {}
173         with self._connect() as conn:
174             rows = (
175                 conn.cursor()
176                 .execute(
177                     "SELECT resource, gpu_type, unit_price_usd FROM pricebook WHERE
version = ?",
178                     (version,),
179                 )
180                 .fetchall()
181             )
182         for r in rows:
183             res, row_gt, price = (
184                 r["resource"],
185                 (r["gpu_type"] or ""),
186                 float(r["unit_price_usd"]),
187             )
188             if res == billing.GPU_SECONDS:
189                 if row_gt == gt: # the GPU rate for THIS gpu_type
190                     rates[res] = price
191             else:
192                 rates[res] = price # wall/egress: gpu_type-independent
193         return rates
194
195     def record_job_cost(
196         self,
197         job_id: str,
198         *,
199         usage: Dict[str, float],
200         pricebook_version: str,
201         gpu_type: Optional[str] = None,
202         compute_backend: Optional[str] = None,
203         owner_id: Optional[str] = None,
204         margin: float = 0.0,
205     ) -> Optional[Dict[str, Any]]:
206         """Compute the job's cost from ``usage`` x the pricebook + persist every cost
column.
207         Returns the stored cost dict, or ``None`` on write failure. The caller gates
this on
208         ``billing.enabled`` (default-OFF) ? nothing records until billing is turned
on."""
209         rates = self.get_pricebook_rates(pricebook_version, gpu_type)
210         # SILENT-UNDER-BILL GUARD: a GPU-seconds AMOUNT with no RATE for this gpu_type
would bill at
211         # $0 indistinguishably from a free run (a new SKU / typo / gpu_type=None).
Surface it loudly +
212         # in the breakdown so the gap is auditable, not invisible. (A missing amount =
$0 IS safe.)
213         gpu_amt = float(usage.get(billing.GPU_SECONDS, 0.0) or 0.0)
214         unpriced_gpu = gpu_amt > 0 and billing.GPU_SECONDS not in rates
215         if unpriced_gpu:
216             logger.warning(
217                 "[BILLING] no pricebook rate for gpu_type=%r in version=%r ? %.1f GPU-
seconds billed "
218                 "at $0 (UNPRICED; recorded in cost_breakdown_json). Add a rate or fix

```

```

gpu_type.",
219         gpu_type,
220         pricebook_version,
221         gpu_amt,
222     )
223     cost_usd, breakdown = billing.compute_cost_usd(usage, rates)
224     if unpriced_gpu:
225         breakdown["unpriced_gpu_seconds"] = gpu_amt
226     if margin:
227         marked = billing.apply_margin(cost_usd, margin)
228         breakdown["margin"] = round(
229             marked - cost_usd, 6
230         ) # keep ?(breakdown) == cost_usd
231         cost_usd = marked
232     bytes_out = int(float(usage.get(billing.EGRESS_GB, 0.0) or 0.0) * 1e9)
233     ok = self._execute_write(
234         "UPDATE jobs SET owner_id=?, compute_backend=?, gpu_type=?,
gpu_active_seconds=?, "
235         "wall_seconds=?, bytes_out=?, gemini_cost_usd=?, pricebook_version=?,
cost_usd=?, "
236         "cost_breakdown_json=? WHERE id = ?",
237         (
238             owner_id,
239             compute_backend,
240             gpu_type,
241             usage.get(billing.GPU_SECONDS),
242             usage.get(billing.WALL_SECONDS),
243             bytes_out,
244             usage.get(billing.GEMINI_USD),
245             pricebook_version,
246             cost_usd,
247             json.dumps(breakdown),
248             job_id,
249         ),
250         f"Failed to record cost for job {job_id}",
251     )
252     if not ok:
253         return None
254     return {
255         "cost_usd": cost_usd,
256         "cost_breakdown": breakdown,
257         "pricebook_version": pricebook_version,
258     }
259
260     def get_job_cost(self, job_id: str) -> Optional[Dict[str, Any]]:
261         """The job's cost row (``None`` if the job doesn't exist; ``cost_usd`` is
262         ``None`` until
263         recorded). ``cost_breakdown`` is the deserialized JSON breakdown."""
264         with self._connect() as conn:
265             row = (
266                 conn.cursor()
267                 .execute(
268                     "SELECT id, video_id, owner_id, compute_backend, gpu_type,
gpu_active_seconds, "
269                     "wall_seconds, bytes_out, gemini_cost_usd, pricebook_version,
cost_usd, "
270                     "cost_breakdown_json FROM jobs WHERE id = ?",
271                     (job_id,),
272                 )
273                 .fetchone()
274             )
275             if not row:
276                 return None
277             d = dict(row)
278             raw = d.pop("cost_breakdown_json", None)

```

```

278         d["cost_breakdown"] = json.loads(raw) if raw else {}
279         return d
280
281     def get_video_cost(self, video_id: str) -> Dict[str, Any]:
282         """Aggregate cost across a video's jobs: total ``cost_usd`` + the per-job rows.
A video is
283         1:1 with an infer job, but a re-ingest can produce several ? sum them."""
284         with self._connect() as conn:
285             rows = (
286                 conn.cursor()
287                 .execute(
288                     "SELECT id, cost_usd, pricebook_version, cost_breakdown_json FROM
jobs "
289                     "WHERE video_id = ?",
290                     (video_id,),
291                 )
292                 .fetchall()
293             )
294         jobs: List[Dict[str, Any]] = []
295         total = 0.0
296         for r in rows:
297             cost = r["cost_usd"]
298             if cost is not None:
299                 total += float(cost)
300             jobs.append(
301                 {
302                     "job_id": r["id"],
303                     "cost_usd": cost,
304                     "pricebook_version": r["pricebook_version"],
305                     "cost_breakdown": json.loads(r["cost_breakdown_json"])
306                     if r["cost_breakdown_json"]
307                     else {},
308                 }
309             )
310         return {"video_id": video_id, "total_cost_usd": round(total, 6), "jobs": jobs}
311
312     # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
-----
313     def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
314         """Attach/replace a job's JSON ExecutionPolicy."""
315         return self._execute_write(
316             "UPDATE jobs SET policy=? WHERE id = ?",
317             (json.dumps(policy) if policy is not None else None, job_id),
318             f"Failed to set policy for job {job_id}",
319         )
320
321     def set_job_waiting(
322         self, job_id: str, delay_sec: float, progress: Optional[str] = None
323     ) -> bool:
324         """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
releasing the
325         worker. ``claim_due_job`` re-picks it when due ? cooperative, no master.
``next_poll_at`` is
326         computed in SQLite's own UTC format so it compares directly to
CURRENT_TIMESTAMP."""
327         return self._execute_write(
328             "UPDATE jobs SET status='waiting', "
329             "next_poll_at=datetime('now', ?), "
330             "progress=COALESCE(?, progress) WHERE id = ?",
331             (f"+{max(0, int(delay_sec))} seconds", progress, job_id),
332             f"Failed to set job {job_id} waiting",
333         )
334
335     def seconds_until_next_due(self) -> Optional[float]:
336         """Seconds until the soonest parked ('waiting') job becomes due, so the worker

```

```

can
337         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
338         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
339         try:
340             with self._connect() as conn:
341                 row = (
342                     conn.cursor()
343                     .execute(
344                         "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
345                         " (julianday(next_poll_at) - julianday('now')) * 86400.0 END)
AS secs "
346                         "FROM jobs WHERE status='waiting'"
347                     )
348                     .fetchone()
349                 )
350             if row is None or row["secs"] is None:
351                 return None
352             return max(0.0, float(row["secs"]))
353         except Exception as e:
354             logger.error(f"Failed to compute next-due delay: {e}")
355             return None
356
357     def claim_due_job(self) -> Optional[Dict[str, Any]]:
358         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
359         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
360         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
361         first. Returns the claimed job dict, or None ONLY when a fresh SELECT finds
nothing
362         runnable ? never for a lost race.
363
364         A lost CAS RETRIES instead of returning: under the parallel drain (#466) several
workers
365         can SELECT the same head-of-queue row, and if the losers reported None the
caller
366         (``_drain_due_jobs``) would read it as "queue empty" and stop with runnable jobs
still
367         queued ? stranding dispatch capacity until some later submit/wake. The retry
loop is
368         deliberately UNBOUNDED: every lost race means another worker successfully
claimed that
369         row, so the runnable set shrank and global progress is guaranteed (lock-free
progress);
370         a retry cap would just reintroduce the false-empty. One connection serves all
attempts ?
371         the SELECT is an autocommit read and each CAS commits before the next iteration,
so no
372         write lock is ever held across a retry (busy_timeout covers momentary
contention)."""
373         try:
374             with self._connect() as conn:
375                 cur = conn.cursor()
376                 while True:
377                     row = cur.execute(
378                         "SELECT id, status FROM jobs WHERE status='queued' "
379                         "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at
<= CURRENT_TIMESTAMP)) "
380                         "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT 1"
381                     ).fetchone()
382                     if not row:
383                         return None # nothing runnable ? a TRUE empty, not a lost race

```

```

384         cur.execute(
385             "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP
386             "
387             "WHERE id=? AND status=?",
388             (row["id"], row["status"]),
389         )
390         conn.commit()
391         if cur.rowcount == 1:
392             break # claimed it
393         # Lost the race: the winner flipped this row; re-SELECT for the next
394         one.
395         return self.get_job(row["id"])
396     except Exception as e:
397         logger.error(f"Failed to claim a due job: {e}")
398         return None
399
400 def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
401     """Crash recovery: any job still queued/running from a previous process is dead
402     (the executor is in-process). Mark failed so clients see a terminal state, not a
403     hang. Returns the number of jobs swept.
404
405     NOTE: ``recover_stale_jobs`` is the preferred startup call ? it RE-QUEUES stale
406     ``compile`` jobs (idempotent CPU stitch, safe to retry) instead of failing them.
407     This
408     blanket-fail method stays for callers/tests that want the old behaviour."""
409     try:
410         with self._connect() as conn:
411             cur = conn.cursor()
412             cur.execute(
413                 "UPDATE jobs SET status='failed', error=?,
414                 finished_at=CURRENT_TIMESTAMP "
415                 "WHERE status IN ('queued', 'running')",
416                 (reason,),
417             )
418             conn.commit()
419             return cur.rowcount
420     except Exception as e:
421         logger.error(f"Failed to sweep stale jobs: {e}")
422         return 0
423
424 def recover_stale_jobs(
425     self, reason: str = "interrupted by server restart"
426 ) -> Tuple[int, int]:
427     """Crash recovery, kind-aware (#329 serving reliability). Called once at
428     startup.
429
430     A ``compile`` job is a **CPU-only, idempotent** ffmpeg re-stitch (re-resolving
431     the same
432     segments yields the same reel), so a compile left queued/running by a killed
433     instance is
434     **re-queued** ? the startup drain re-runs it and the client keeps polling until
435     it gets
436     the reel, instead of seeing a spurious "failed". A ``process`` job is an
437     expensive,
438     non-idempotent GPU detection pass; auto-retrying could double-spend, so it is
439     **failed**
440     (terminal, as before) and the client re-submits deliberately.
441
442     Returns ``(requeued_compile, failed_process)``."""
443     try:
444         with self._connect() as conn:
445             cur = conn.cursor()
446             # Re-queue interrupted COMPILE jobs (idempotent ? safe to retry
447             automatically).
448             cur.execute(

```

```

438             "UPDATE jobs SET status='queued', started_at=NULL,
next_poll_at=NULL, "
439             "error=NULL WHERE status IN ('queued', 'running') AND
kind='compile'"
440         )
441         requeued = cur.rowcount
442         # Fail interrupted PROCESS jobs (GPU, not safe to auto-retry).
443         cur.execute(
444             "UPDATE jobs SET status='failed', error=?,
finished_at=CURRENT_TIMESTAMP "
445             "WHERE status IN ('queued', 'running') AND (kind IS NULL OR kind !=
'compile')",
446             (reason,),
447         )
448         failed = cur.rowcount
449         conn.commit()
450         return requeued, failed
451     except Exception as e:
452         logger.error(f"Failed to recover stale jobs: {e}")
453         return 0, 0
454
455

```

12. user (2026-07-04T11:49:36.372Z)

```

1      """Path-safety helpers for the API boundary (untrusted request fields)."""
2
3      import os
4      import re
5      from typing import Any, Optional
6
7
8      def output_base(config: Any) -> str:
9          """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
10             (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
11             chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
12             a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
13
14             This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
15             ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
16             (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``. """
17             try:
18                 out = (
19                     config.output.output_dir
20                     if not isinstance(config, dict)
21                     else config.get("output", {}).get("output_dir")
22                 )
23             except Exception:
24                 out = None
25             return out or "output"
26
27
28      def safe_output_filename(name: str) -> str:
29          """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
30
31             ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
absolute,
32             and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-

```

```

write
33     primitive. Returns the name unchanged when safe; raises ValueError otherwise.
34     """
35     if not name or not name.strip():
36         raise ValueError("output_filename must not be empty")
37     if "\x00" in name:
38         raise ValueError("output_filename must not contain a null byte")
39     if "/" in name or "\\" in name:
40         raise ValueError(f"output_filename must not contain a path separator: {name!r}")
41     if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
42         raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
43     return name
44
45
46     def resolve_under_root(path: str, root: str) -> str:
47         """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
48         rp = os.path.realpath(path)
49         rr = os.path.realpath(root)
50         try:
51             contained = os.path.commonpath([rp, rr]) == rr
52         except ValueError: # different drives on Windows
53             contained = False
54         if not contained:
55             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
56         return rp
57
58
59     # A storage URI (gs://, s3://, gdrive://, ?) starts with a scheme; it is NOT a local
filesystem path,
60     # so os.path.realpath() would mangle it into a bogus cwd-relative path ? the local-path
containment
61     # jail can't be applied to it. A bucket URI is instead scoped to the configured
input_root (below).
62     _URI_SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
63
64
65     def _uri_under_root(uri: str, root: str) -> bool:
66         """True if the storage URI `uri` is exactly `root` or a child of it ? a
normalized-prefix check
67         with a separator boundary, so `gs://b/videos-evil` is NOT under
`gs://b/videos`."""
68         u = str(uri).rstrip("/")
69         r = str(root).rstrip("/")
70         return bool(r) and (u == r or u.startswith(r + "/"))
71
72
73     def validate_input_path(
74         path: str,
75         allowed_root: Optional[str] = None,
76         *,
77         allowed_uri_root: Optional[str] = None,
78     ) -> None:
79         """Reject null bytes and (in hosted mode) out-of-bounds inputs.
80
81         With no `allowed_root` (default, single-user local mode) only the null-byte check
applies, so the
82         local 'point it at any of my videos' workflow is preserved.
83
84         Setting `allowed_root` (a hosted/tenant deployment) enforces containment so an
authenticated
85         tenant can't point the engine at data it shouldn't reach:
86         - a LOCAL path must resolve under `allowed_root` (a realpath containment jail);
and
87         - a storage URI (gs://, s3://, gdrive://) ? which realpath can't jail ? is allowed

```



```

ONLY when it is
88         under ``allowed_uri_root`` (the configured ``storage.input_root`` bucket
prefix). An arbitrary
89         bucket, or any URI when no ``allowed_uri_root`` is configured, is REJECTED
(tenant isolation).
90
91     The null-byte check always applies.
92     """
93     if "\x00" in path:
94         raise ValueError("video_path must not contain a null byte")
95     if not allowed_root:
96         return
97     if _URI_SCHEME_RE.match(path):
98         if _uri_under_root(path, allowed_uri_root or ""):
99             return
100         raise ValueError(
101             f"hosted mode: {path!r} is not under the allowed input root "
102             f"{(allowed_uri_root or '')!r}"
103         )
104     resolve_under_root(path, allowed_root)
105

```

13. user (2026-07-04T11:49:37.674Z)

```

22-class GCSStorage(StorageBackend):
23-    def __init__(
24-        self,
25-        *,
26-        client: Any = None,
27-        project: Optional[str] = None,
28-        signed_url_ttl: int = 3600,
29-        **_ignored,
30-    ) -> None:
31-        self._ttl = signed_url_ttl
32-        if client is not None:
33-            self._client = client
34-            return
35-        storage_module: Any = import_module("google.cloud.storage")
36-        self._client = storage_module.Client(project=project)
37-
38-    --
39-
40-        "gcs", prefix.bucket, blob.name, f"gcs://{prefix.bucket}/{blob.name}"
41-    )
42-
43-    def delete(self, ref) -> None:
44-        self._blob(ref).delete()
45-
46-    def signed_url(self, ref, *, ttl=None, method="GET") -> str:
47-        blob = self._blob(ref)
48-        exp = timedelta(seconds=ttl or self._ttl)
49-        creds = getattr(self._client, "_credentials", None)
50-        # Local-key signing (a service-account JSON key exposes a `signer`) works out of the
51-        box. But
52-        # on Cloud Run / GCE the ADC credentials come from the metadata server and have NO
53-        private key,
54-        # so a plain generate_signed_url raises "you need a private key". Fall back to the
55-        IAM signBlob
56-        # API ? which requires the runtime SA to hold roles/iam.serviceAccountTokenCreator on
57-        ITSELF.
58-        # ? signBlob canNOT ride the storage client's own token (#475, found on the first
59-        real-creds
60-        # run): that token is scoped to devstorage.* only (Client.SCOPE), which
61-        iamcredentials rejects
62-        # with ACCESS_TOKEN_SCOPE_INSUFFICIENT ? and a metadata credential may still report
63-        the

```

```

90-         # "default" email alias. Mint a separate cloud-platform-scoped credential instead:
its refresh
91-         # both authorizes signBlob and resolves the real SA email. Failures are LOGGED before
deferring
92-         # to the default call, so a broken sign path can't masquerade as a missing-key error.
93-         if creds is not None and not hasattr(creds, "signer"):
94-             try:
95-                 sa_email, token = self._signblob_identity()
96-                 if sa_email:
97-                     return blob.generate_signed_url(
98-                         version="v4",
99-                         expiration=exp,
100-                        method=method,
101-                        service_account_email=sa_email,
102-                        access_token=token,
103-                    )
104-                 logger.warning(
105-                     "[gcs] keyless signing skipped: runtime SA email unresolved (got %r); "
106-                     "falling back to key-based signing",
107-                     sa_email,
108-                 )
109-             except Exception as e:
110-                 logger.warning(
111-                     "[gcs] keyless signBlob path failed (%s); falling back to key-based
signing",
112-                     e,
113-                 )
114-         return blob.generate_signed_url(version="v4", expiration=exp, method=method)
115-
116-     @staticmethod
117-     def _signblob_identity() -> "tuple[Optional[str], Optional[str]]":
118-         """(sa_email, access_token) for the IAM signBlob call ? a FRESH cloud-platform-
scoped
119-         credential from ADC, never the storage client's devstorage-scoped token (#475).
Isolated
120-         as a seam so tests can fake the metadata-server identity without real
credentials."""
121-         import google.auth
122-         from google.auth.transport.requests import Request as _AuthRequest
123-
124-         sign_creds, _ = google.auth.default(
125-             scopes=["https://www.googleapis.com/auth/cloud-platform"]
126-         )
127-         sign_creds.refresh(_AuthRequest())
128-         sa_email = getattr(sign_creds, "service_account_email", None)
129-         if not sa_email or sa_email == "default":
130-             return None, None
131-         return sa_email, sign_creds.token

```

14. user (2026-07-04T11:49:45.631Z)

```

1     """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3     Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4     - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5     for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6     so it is abandoned, not joined ? it never blocks the process.
7     - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8     ``max_wait_sec`` deadline).
9
10    **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-

```

```

tested with zero real
11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17
18     from __future__ import annotations
19
20     import logging
21     import threading
22     import time as _time
23     from dataclasses import asdict, dataclass
24     from enum import Enum
25     from typing import Callable, Optional, TypeVar
26
27     T = TypeVar("T")
28
29     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
30     LOG = logging.getLogger("execution.policy")
31
32
33     class WaitMode(str, Enum):
34         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
35
36         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
37         ABORT = "abort" # one attempt; on hang/fail, stop immediately
38         BLOCK = "block" # bounded in-process blocking wait (no reschedule)
39
40
41     class PolicyTimeout(TimeoutError):
42         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
43
44
45     @dataclass(frozen=True)
46     class ExecutionPolicy:
47         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
48
49         mode: WaitMode = WaitMode.WAIT_AND_RESUME
50         poll_every_n_sec: float = 10.0
51         op_timeout_sec: float = (
52             120.0 # HARD per-attempt deadline ? the hang-killer that was missing
53         )
54         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
55         max_retries: int = 5 # retries (=> max_retries+1 attempts) for failures/hangs
56         backoff_base_sec: float = 3.0 # exponential: backoff_base * 2**attempt
57         on_timeout: str = "reschedule" # reschedule | partial | fail (consumed by P2/P3)
58         resumable: bool = True
59
60         def to_dict(self) -> dict:
61             d = asdict(self)
62             d["mode"] = self.mode.value
63             return d
64
65         @classmethod
66         def from_dict(cls, d: dict) -> "ExecutionPolicy":
67             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]

```

```

68         kw = {k: v for k, v in d.items() if k in known}
69         if "mode" in kw and not isinstance(kw["mode"], WaitMode):
70             kw["mode"] = WaitMode(kw["mode"])
71         return cls(**kw)
72
73
74     #: The sane default (also what a job gets if it declares no policy).
75     DEFAULT_POLICY = ExecutionPolicy()
76
77
78     def _backoff_sec(policy: ExecutionPolicy, attempt: int) -> float:
79         """Deterministic exponential backoff (no jitter ? reproducible tests)."""
80         return policy.backoff_base_sec * (2**attempt)
81
82
83     def run_bounded(
84         fn: Callable[[], T],
85         policy: ExecutionPolicy = DEFAULT_POLICY,
86         *,
87         label: str = "op",
88         logger: Optional[logging.Logger] = None,
89         sleep: Callable[[float], None] = _time.sleep,
90     ) -> T:
91         """Run ``fn()`` under a hard per-attempt timeout, retrying FAILURES and HANGS with
92         backoff.
93
94         Returns ``fn``'s value on success. Raises ``PolicyTimeout`` if the final attempt
95         hung, or re-raises the
96         last exception if the final attempt failed. ``mode=ABORT`` => a single attempt (no
97         retries).
98
99         """
100         log = logger or LOG
101         attempts = 1 if policy.mode is WaitMode.ABORT else policy.max_retries + 1
102         last_exc: Optional[BaseException] = None
103         for attempt in range(attempts):
104             box: dict = {}
105             done = threading.Event()
106
107             # box/done bound as defaults (not closed over) ? the thread is started + joined
108             # same iteration, but explicit binding keeps each attempt isolated and silences
109             B023.
110
111             def _runner(_box: dict = box, _done: threading.Event = done) -> None:
112                 try:
113                     _box["val"] = fn()
114                 except (
115                     BaseException
116                 ) as e: # relay ANY error (incl. timeouts) to the caller thread
117                     _box["exc"] = e
118                 finally:
119                     _done.set()
120
121             threading.Thread(target=_runner, name=f"bounded:{label}", daemon=True).start()
122             if done.wait(timeout=policy.op_timeout_sec):
123                 if "exc" in box:
124                     last_exc = box["exc"]
125                     log.info(
126                         "%s: attempt %d/%d failed: %s",
127                         label,
128                         attempt + 1,
129                         attempts,
130                         last_exc,
131                     )
132                 else:
133                     if attempt:

```

```

128         log.info(
129             "%s: succeeded on attempt %d/%d", label, attempt + 1, attempts
130         )
131         return box["val"] # type: ignore[return-value]
132     else:
133         last_exc = PolicyTimeout(
134             f"{label} hung > op_timeout_sec={policy.op_timeout_sec}s"
135         )
136         log.warning(
137             "%s: attempt %d/%d HUNG (>%.0fs) ? abandoning",
138             label,
139             attempt + 1,
140             attempts,
141             policy.op_timeout_sec,
142         )
143         if attempt < attempts - 1:
144             wait = _backoff_sec(policy, attempt)
145             log.debug("%s: backing off %.1fs before retry", label, wait)
146             sleep(wait)
147         assert last_exc is not None
148         log.warning("%s: gave up after %d attempt(s): %s", label, attempts, last_exc)
149         raise last_exc
150
151
152     def poll_until(
153         check: Callable[[], Optional[T]],
154         policy: ExecutionPolicy = DEFAULT_POLICY,
155         *,
156         label: str = "op",
157         logger: Optional[logging.Logger] = None,
158         sleep: Callable[[float], None] = _time.sleep,
159         monotonic: Callable[[], float] = _time.monotonic,
160     ) -> T:
161         """Poll ``check()`` every ``poll_every_n_sec`` until it returns a non-``None``
162         value, or raise
163         ``PolicyTimeout`` after ``max_wait_sec``. ``check`` returns ``None`` while the op is
164         still in progress.
165
166         Polls log at DEBUG (no INFO spam); the wait start + resolution + give-up log at
167         INFO/WARNING.
168         """
169         log = logger or LOG
170         start = monotonic()
171         polls = 0
172         log.info(
173             "%s: waiting (poll_every=%.0fs, max_wait=%.0fs)",
174             label,
175             policy.poll_every_n_sec,
176             policy.max_wait_sec,
177         )
178         while True:
179             result = check()
180             if result is not None:
181                 log.info(
182                     "%s: ready after %d poll(s) / %.0fs", label, polls, monotonic() - start
183                 )
184                 return result
185             elapsed = monotonic() - start
186             if elapsed >= policy.max_wait_sec:
187                 log.warning(
188                     "%s: still not ready after %.0fs (max_wait) ? giving up", label, elapsed
189                 )
190                 raise PolicyTimeout(
191                     f"{label} not ready within max_wait_sec={policy.max_wait_sec}s"
192                 )

```

```

190         polls += 1
191         log.debug("%s: poll %d, %.0fs elapsed, not ready", label, polls, elapsed)
192         sleep(policy.poll_every_n_sec)
193

```

15. user (2026-07-04T11:49:45.951Z)

```

1     """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3     **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4     ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5     ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6     signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spooof** the
7     identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8     token's ``email``, else ``sub``).
9
10    ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11    ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is
12    unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13    """
14
15    from __future__ import annotations
16
17    import json
18    import time
19    import urllib.request
20    from typing import Any, Callable, Mapping, Optional
21
22    # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23    CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24    _JWKS_PATH = "/cdn-cgi/access/certs"
25    _JWKS_TTL_SEC = (
26        600.0 # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
27    )
28
29    JwksFetcher = Callable[[str], dict]
30
31
32    class EdgeAuthError(Exception):
33        """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
34
35
36    # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
37    _jwks_cache: dict[str, tuple[float, dict]] = {}
38
39
40    def _issuer(team_domain: str) -> str:
41        return team_domain.rstrip("/")
42
43
44    def _default_jwks_fetcher(team_domain: str) -> dict:
45        """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ?
only the

```

```

46     production path reaches here; tests pass an injected fetcher."""
47     url = _issuer(team_domain) + _JWKS_PATH
48     with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
49         data: Any = json.load(resp)
50     if not isinstance(data, dict) or "keys" not in data:
51         raise EdgeAuthError(f"unexpected JWKS shape from {url}")
52     return data
53
54
55 def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
56     """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher``
(tests)
57     bypasses the cache so each test is deterministic."""
58     if fetcher is not None:
59         return fetcher(team_domain)
60     now = time.time()
61     cached = _jwks_cache.get(team_domain)
62     if cached and (now - cached[0]) < _JWKS_TTL_SEC:
63         return cached[1]
64     jwks = _default_jwks_fetcher(team_domain)
65     _jwks_cache[team_domain] = (now, jwks)
66     return jwks
67
68
69 def verify_cf_access_token(
70     token: str, *, team_domain: str, aud: str, jwks: dict
71 ) -> str:
72     """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
73     return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
74     try:
75         import jwt
76         from jwt import PyJWKSet
77     except (
78         Exception
79     ) as exc: # pragma: no cover - only when the auth extra isn't installed
80         raise EdgeAuthError(
81             "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
82             "(pip install -e .[auth]) on a host with edge_auth_enabled."
83         ) from exc
84
85     try:
86         kid = jwt.get_unverified_header(token).get("kid")
87         if not kid:
88             raise EdgeAuthError("token header has no kid")
89         signing_key = PyJWKSet.from_dict(jwks)[kid]
90         claims = jwt.decode(
91             token,
92             signing_key.key,
93             algorithms=["RS256"],
94             audience=aud,
95             issuer=_issuer(team_domain),
96             options={"require": ["exp", "iss", "aud"]},
97         )
98     except EdgeAuthError:
99         raise
100     except KeyError as exc: # kid not in the JWKS
101         raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
102     except (
103         Exception
104     ) as exc: # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
105         raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
106

```

```

107     user = claims.get("email") or claims.get("sub")
108     if not user:
109         raise EdgeAuthError("verified token carries no email/sub claim")
110     return str(user)
111
112
113     def resolve_tenant(
114         headers: Mapping[str, str],
115         config: Any,
116         *,
117         jwks_fetcher: Optional[JwksFetcher] = None,
118     ) -> Optional[str]:
119         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
120
121         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
122         user; unchanged).
123         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
124         on any miss.
125         ``jwks_fetcher`` is injectable for offline tests.
126         """
127         sec = getattr(config, "security", None)
128         if not sec or not getattr(sec, "edge_auth_enabled", False):
129             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
130
131         # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure
132         check in
133         # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured",
134         both places.
135         team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
136         aud = (getattr(sec, "cf_access_aud", "") or "").strip()
137         if not team_domain or not aud:
138             raise EdgeAuthError(
139                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
140                 configured"
141             )
142
143         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
144         support a plain
145         # dict too (tests) by trying both casings.
146         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
147         if not token:
148             raise EdgeAuthError(
149                 f"missing {CF_ACCESS_HEADER} header (request did not pass Cloudflare
150                 Access)"
151             )
152
153         jwks = _get_jwks(team_domain, jwks_fetcher)
154         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
155
156

```

16. user (2026-07-04T11:49:52.516Z)

```

1     # A1 alpha serving ? post-merge e2e deploy test (cloud-serving)
2
3     **Date:** 2026-07-04 . **Region:** asia-southeast1 . **Project:** gen-lang-
4     client-0306026826
5     **Scope:** A1 (ALPHA_LAUNCH_READINESS §7) closure ? prove the merged serving fixes
6     (#465 input jail, #467 upload staging, #468 drain concurrency, #469 signed-URL download,
7     #470 auth-in-base-image) plus the committed manifest generator (#474) on a REAL deployed
8     build,
9     then restore the live edge-auth posture.
10
11     > Companion to `DEPLOY_REPORT_cloud-serving_a1-cuj_2026-07-04.md` (PR #464): that report
12     stood the
13     > stack up and FILED #461/#463/#466; THIS report proves their fixes end-to-end on merged

```



```

master and
11     > records the one new bug the run surfaced (#475, fixed by #476 mid-run).
12
13     ## 1. What was deployed
14
15     | Resource | Value |
16     |---|---|
17     | Source | merged `master` ? `195932f` (first build) ? **`3c97117`** (final build, incl.
#476) |
18     | Worker image | `?/rally/rally-worker:latest` ? build 1 `sha256:ed2fe448?` (195932f),
build 2 **`sha256:31475f27?`** (3c97117); the Job re-pinned via `gcloud run jobs update` after
each build |
19     | L4 worker **Job** | `rally-worker` ? unchanged shape (nvidia-l4, 8 vCPU/32Gi, FUSE
`khelsutra-rally-serving`?`/gcs`) |
20     | Control-plane **Service** | `rally-control-plane` ? rev **00005** (auth-off, build 1)
? rev **00006** (auth-off, final digest) ? rev **00007** (auth-ON restore, final digest).
Deployed FROM THE BASE IMAGE directly ? no derived PyJWT overlay (**#470 proven**); `maxScale=1`
(#471 option 1) |
21     | Manifest | generated by the **committed** `deploy/cloudrun/gen_service_yaml.py` (#474)
? `--edge-auth off` for the test window, `--edge-auth on` to restore. The scratchpad-artifact
gap (#473) is closed: this deploy used only repo-committed tooling |
22
23     ## 2. Boot / health (rev 00005+, identity-token auth)
24
25     - `GET /api/health` ? `200 {"status":"ok",?}`; `GET /api/capabilities` ? `wasb_hybrid`
present,
26     `gemini_key_present=false` (skip_ai_handoff honored), ffmpeg/ffprobe present.
27     - Before-state captured on the outgoing rev 00004: `/api/process` without a CF JWT ?
**401**
28     `edge auth: missing Cf-Access-Jwt-Assertion header` ? the M9 posture was live as
documented.
29
30     ## 3. CUJ on the FINAL image (rev 00006, digest `31475f27?`) ? all green
31
32     1. **gs:// process** ? `POST /api/process {gs://?/MahadevpuraSingles.MP4, wasb_hybrid,
33     compute=cloud}` ? 202 ? `validating ? dispatching (cloud_run) ? done` ? **48 play
segments**
34     (same count as the #464 run; job `18dc6502?`).
35     2. **#461 upload staging** ? chunked upload (`init ? part/0 ? complete`) of a
105,345-byte 720p
36     clip ? `POST /api/process` with the returned **local** path + `compute=cloud` ? the
job staged
37     it to `gs://khelsutra-rally-serving/videos/d8335f51?/upload-e2e-720p.mp4` (byte-exact
size
38     verified in the bucket) and dispatched to the L4 ? **success** (0 segments ? correct
for a
39     synthetic test pattern). NOT the pre-fix "cloud-serving needs a cloud input" hard-
fail.
40     Verified on BOTH builds (`ed2fe448?` and the final `31475f27?`).
41     - ? Upload validation floor: a 640x360 test clip is rejected (`resolution_fps_check`,
42     min 1280x720) ? the first attempt failed on this; alpha docs should mention the
floor.
43     3. **#468 drain concurrency, observed live** ? the upload job staged + dispatched
**while** the
44     48-segment reel compile was stitching on the CP: remote dispatch overlapped the local
compile
45     lane exactly as designed (no serialization, no false-empty stall). On rev 00006 the
two process
46     jobs (gs:// + upload) also ran the drain concurrently.
47     4. **compile** ? `POST /api/compile {video_id, segment_ids:[1..48], "e2e.mp4"}` ? 202 ?
done;
48     measured stitch wall on rev 00005: created 10:39:32 ? finished 10:44:26 (**~4m54s**,
incl. the
49     gs:// source re-fetch); rev 00006 re-run equally green. Reel **mirrored to
50     `gs://khelsutra-rally-serving/highlights/915473?/e2e.mp4` ? 174,046,228 bytes (~166

```

```

MiB)**.
51     5. **#463 signed-URL download ? VERIFIED**:
52     - `GET /api/highlights/915473?/e2e.mp4` ? **`307`** ?
53     - `https://storage.googleapis.com/khelsutra-rally-
serving/highlights/?/e2e.mp4?X-Goog-Algorithm=G00G4-RSA-SHA256&X-Goog-
Credential=492532266377-compute@?&X-Goog-Expires=3600&X-Goog-Signature=?`
54     (v4, signed AS the runtime SA via keyless signBlob ? no key file anywhere).
55     - `curl -L` through the redirect: **full 174,046,228 bytes in 8.1s** (single GET ? no
32 MiB
56     edge-500, no Range workaround), and the file **ffprobes valid**: H.264 1920×1080 @
29.97 fps
57     + AAC, duration 244.29s (the 48-rally reel).
58
59     ## 4. Bug surfaced mid-run: #475 ? fixed by #476 (merged, redeployed)
60
61     On the FIRST attempt (rev 00005, build `ed2fe448?` = pre-#476), the #463 check
**failed**:
62     `GET /api/highlights/...` ? **500**; log:
63     `[highlights] signed-url path failed ? : you need a private key to sign credentials`.
64
65     - The reel WAS mirrored correctly; the **keyless signBlob path** failed and the handler
fell back
66     to serving the 166 MiB file locally ? Cloud Run's ~32 MiB cap ? edge-500 (the original
#463
67     symptom, resurrected by a different root cause).
68     - **Root cause (verified live):** `GCSStorage.signed_url()` reused the storage client's
own token
69     for `iamcredentials.signBlob`; `google.cloud.storage.Client.SCOPE` is **devstorage-
only**, which
70     signBlob rejects (`ACCESS_TOKEN_SCOPE_INSUFFICIENT`) ? and the branch's bare `except:
pass`
71     swallowed the real error, so the fallback raised the misleading private-key
message.
72     Environment was clean: the SA holds self-`serviceAccountTokenCreator` and
73     `iamcredentials.googleapis.com` is enabled ? code bug, not config.
74     - **Why tests never saw it:** the fakes carry no `_credentials`; this was the FIRST run
of #469's
75     path on real compute credentials. Filed **#475** ? fixed by **#476**
(`_signblob_identity`)
76     mints a fresh `cloud-platform`-scoped ADC credential; failures now LOGGED before
falling back;
77     regression tests reproduce the credential shape). Suite 1659? / cov 85.46% / ruff+mypy
clean;
78     merged on green CI per the standing agreement; rebuilt (`31475f27?`) and redeployed
(rev 00006);
79     $3.5 is the pass on the fixed build.
80
81     ## 5. Notes / observations (rough edges captured)
82
83     - **`services replace` with an unchanged manifest is a NO-OP** ? it does not re-resolve
`:latest`
84     after a rebuild. Force a new revision (`gcloud run services update --image ? :latest`)
or stamp a
85     revision suffix. Follow-up candidate: `--revision-suffix` in `gen_service_yaml.py`.
86     - **A redeploy wipes the per-instance SQLite state** ? `/api/highlights` then 404s on
the DB
87     record check (`backend/main.py:1052`) even though the reel mirror exists in gs://.
Known, filed
88     as **#471**; the CUJ was re-run on rev 00006 to regenerate state. Alpha-operator
implication: a
89     deploy invalidates in-flight jobs and past download URLs (re-process regenerates
them).
90     - Upload validation floor (?1280×720) ? $3.2.
91     - 422-before-401 quirk: a **malformed-JSON** body gets FastAPI's 422 before the edge-
auth

```

```

92      dependency runs; any well-formed request without a CF JWT still 401s. No exposure;
noted so a
93      422 in edge-auth triage isn't misread.
94
95      ## 6. Edge-auth restore + live posture (rev 00007, digest `31475f27?`)
96
97      - Redeployed with the generator's `--edge-auth on` manifest ? rev **00007**.
98      - `GET /api/health` ? **200** (deliberately exempt).
99      - `POST /api/process` (valid JSON, no CF JWT) ? **401**
100      `edge auth: missing Cf-Access-Jwt-Assertion header`.
101      - `https://api.khelsutra.guru/api/health` (no CF session) ? **302** ?
102      `https://khelsutra.cloudflareaccess.com/cdn-cgi/access/login/api.khelsutra.guru??`
with the
103      Access-app aud `c980da67?082b` in the login meta ? the exact pre-test posture,
restored.
104
105      ## 7. A1 DoD checklist
106
107      | DoD item | Status |
108      |---|---|
109      | Serving endpoint deployed + healthy | ? rev 00007, final digest `31475f27?` |
110      | Upload ? process ? jobs ? compile ? download CUJ | ? end-to-end on the final image |
111      | Edge-auth (M9) restored + verified | ? 401 sans JWT; api.khelsutra.guru ? CF login 302
|
112      | >32 MiB download via signed URL (#463) | ? 307 + full 174 MB single-GET + ffprobe-
valid |
113      | Browser-upload staging to gs:// (#461) | ? staged byte-exact + dispatched + success |
114      | Parallel drain (#466/#468) observed live | ? dispatch overlapped compile; 2 process
jobs drained concurrently |
115
116      **A1 ? DONE** (flipped in `docs/ALPHA_LAUNCH_READINESS.md` §7 in this PR).
117

```

17. user (2026-07-04T11:50:04.664Z)

```

109:| A10 | Decide and record the alpha ship-gate target: metric, corpus slice, minimum
threshold, significance rule, and what claims are allowed | Owner + `badminton-highlight-
indexer` | A4, A5, A8 | Owner decision | GATED | - |
110:| A11 | Resolve or explicitly defer WASB C3 provenance for alpha messaging; if deferred,
document "no commercial model sharing" copy guardrail | Owner/legal | A10 | Legal/provenance |
GATED | - |
112:| A13 | Alpha launch review: runbook, privacy/consent copy, observability/job-failure
triage, known limits, rollback/disable steps, and owner GO/NO-GO | Cross-repo | A1, A4, A10, A11
| Owner launch decision | TODO | - |

```

18. user (2026-07-04T11:50:09.288Z)

```

1      ""Free-space guards for the local box (STUDIO_MEDIA_LIFECYCLE.md P2, D11).
2
3      The single-box appliance fills its disk in three peaks: a chunked **upload** (raw file
into
4      ``upload_dir``), a **fetch-to-scratch** of a bucket/Drive source (a full local copy
under the temp
5      dir), and ? the real balloon ? **WSL frame extraction**, which can materialise ~15× the
source in
6      PNGs. D11: BLOCK on a local/WSL shortfall (buckets are warn-only elsewhere ? they aren't
capacity-bounded from here).
7
8
9      ``require_free_bytes`` is the HTTP-boundary guard: it raises **HTTP 507 (Insufficient
Storage)**
10     when the filesystem holding ``path`` can't hold ``need`` bytes, so the client fails fast
instead of
11     crashing a job mid-flight. It is best-effort about *measuring* (an unreadable
``disk_usage`` never
12     blocks ? inability to measure must not deny a legitimate request), but strict once a

```

```

real shortfall
13     is seen. The WSL frame-stage guard lives on ``wslCommandMixin`` (a different filesystem,
measured via
14     ``df`` inside WSL, not ``shutil.disk_usage``) ? see ``pipeline/detectors/base.py``.
15     """
16
17     from __future__ import annotations
18
19     import logging
20     import shutil
21     from typing import Optional
22
23     from fastapi import HTTPException
24
25     logger = logging.getLogger(__name__)
26
27
28     def free_bytes(path: str) -> Optional[int]:
29         """Best-effort free bytes on the filesystem holding ``path`` (``None`` if
undeterminable).
30
31         Never raises. ``shutil.disk_usage`` wants an existing path; we walk up to the
nearest existing
32         ancestor (an ``upload_dir``/scratch target may not exist yet at guard time)."""
33         import os.path
34
35         probe = path
36         for _ in range(64): # bounded ancestor walk (defensive against a pathological path)
37             if not probe:
38                 return None
39             try:
40                 return int(shutil.disk_usage(probe).free)
41             except (OSError, ValueError):
42                 parent = os.path.dirname(probe.rstrip("/\\"))
43                 if parent == probe: # reached the root and still can't stat ? give up
44                     return None
45                 probe = parent
46         return None
47
48
49     def require_free_bytes(
50         path: str, need: Optional[int], *, stage: str = "write"
51     ) -> None:
52         """Raise HTTP 507 if ``path``'s filesystem can't hold ``need`` bytes; otherwise
return.
53
54         ``need`` of ``None`` or ``<= 0`` is a no-op (nothing to reserve). Measuring is best-
effort: an
55         undeterminable free-space read logs a warning and proceeds (we never deny on
inability to
56         measure). A confirmed shortfall raises so the caller fails fast with an actionable
507."""
57         if not need or need <= 0:
58             return
59         free = free_bytes(path)
60         if free is None:
61             logger.warning(
62                 "free-space guard (%s): could not read free space at %s ? proceeding.",
63                 stage,
64                 path,
65             )
66             return
67         if free < need:
68             need_mb = need / (1024 * 1024)
69             free_mb = free / (1024 * 1024)

```

```

70         logger.warning(
71             "free-space guard (%s): %s needs %.0f MB but only %.0f MB free ? refusing
(507).",
72             stage,
73             path,
74             need_mb,
75             free_mb,
76         )
77         raise HTTPException(
78             status_code=507,
79             detail=(
80                 f"Not enough free disk space for {stage}: need ~{need_mb:.0f} MB, "
81                 f"only ~{free_mb:.0f} MB free. Free some space and retry."
82             ),
83         )
84

```

19. user (2026-07-04T11:50:19.423Z)

```

326
Hosted-app observability: request tracing + replay for debugging production CUJ failures
OPEN
328
Surface the per-video runlog / execution trace to the user (endpoint + UI), not just server logs
OPEN

```

20. user (2026-07-04T11:50:20.084Z)

```

__init__.py
__main__.py
__pycache__
134: def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
136:     outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
139:     A remote job that never writes a completion marker (a failed/hung container) makes the
bucket
154:         "[worker] remote compute did not complete (no marker within budget): %s", e
158:         "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
166: def _write_done_marker(
167:     done_uri: str,
175:     """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer) for a
remote
178:         marker = {
184:         local = os.path.join(scratch, "_done.json")
186:         json.dump(marker, f)
187:         storage.put(local, done_uri, config=storage_cfg)
188:         logger.info("[worker] wrote completion marker -> %s", done_uri)
189:     except Exception as e: # never fail a good run because the marker push hiccuped
190:         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
205:         return _dispatch_remote(compute, args)
236:         # M6: on a worker-side FAILURE, write the done-marker (the REAL rc, 0 segments) so a
remote
238:         # budget expires. DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
239:         if getattr(args, "done_uri", None):
240:             _write_done_marker(
241:                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
327:         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushes)}
artifact(s):"
332:         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so its
bucket-poll
334:         # write their own marker via _fail() above, so the dispatcher fails FAST either way (no
30-min hang
335:         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local
run).

```

```

336:     if getattr(args, "done_uri", None):
337:         _write_done_marker(
338:             args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
622:     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
661:         "--done-uri",

```

21. user (2026-07-04T11:50:35.678Z)

```

465     MERGED   fix(hosted): allowed_video_root jail must not reject storage URIs (gs://)
476     MERGED   fix(storage): GCS keyless signing mints a cloud-platform-scoped token for
signBlob (closes #475)
471     OPEN     cloud-serving: control-plane state (SQLite jobs/videos DB) is per-instance
ephemeral ? maxScale>1 splits the queue and /api/jobs polls
461     CLOSED   cloud-serving: browser-uploaded local files not staged to gs:// before L4
dispatch (upload CUJ fails)
463     CLOSED   cloud-serving: /api/highlights download 500s for reels >32 MiB (Cloud Run
response limit)

```

22. user (2026-07-04T11:51:28.388Z)

Found 2 files
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\eval_partial_labels.py
C:\Users\avidu\Projects\khehsutra-guru\badminton-highlight-indexer\backend\eval\export_wasb_segments.py

23. user (2026-07-04T11:51:29.126Z)

```

36:     "acquire_local_input",
76: def acquire_local_input(
103:     """Delete a remote scratch dir produced by ``acquire_local_input``.

```

24. user (2026-07-04T11:52:27.868Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
``None``
5     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
behaviour, byte
6     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
``cpu_mock`` nothing
7     changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
needed).
8     ""
9
10    from __future__ import annotations
11
12    import os
13    from typing import Any, Optional
14
15    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
16    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
17
18    __all__ = [
19        "ComputeBackend",
20        "ComputeJobSpec",
21        "ComputeResult",
22        "get_compute_backend",
23    ]
24
25
26    def _storage_dict(config: Any) -> dict:

```

```

27     storage = getattr(config, "storage", None)
28     if storage is None:
29         return {}
30     if hasattr(storage, "model_dump"):
31         return storage.model_dump() # type: ignore[no-any-return]
32     return dict(storage) if isinstance(storage, dict) else {}
33
34
35     def get_compute_backend(
36         config: Any,
37         *,
38         run_client: Optional[Any] = None,
39         policy: ExecutionPolicy = DEFAULT_POLICY,
40         token: Optional[str] = None,
41         target_override: Optional[str] = None,
42     ) -> Optional[ComputeBackend]:
43         """Return the compute backend for ``config.deployment.compute_target``, or ``None``
44         to run
45         in-process (the default-OFF path).
46
47         Only ``compute_target == "cloud_run"`` returns a non-None backend (a
48         ``CloudRunCompute``). Its
49         Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
50         ``CLOUD_RUN_REGION``
51         / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
52         injectable (tests
53         pass a fake; production defaults to the real ``google-cloud-run`` client, owner-
54         verified on M7).
55
56         ``target_override`` lets a single request force a target (the SPA compute-picker
57         maps a
58         per-request ``compute="cloud"`` to ``target_override="cloud_run"``) without mutating
59         the
60         server's configured default. ``None`` falls back to
61         ``config.deployment.compute_target``.
62
63         """
64         target = (
65             target_override
66             if target_override is not None
67             else getattr(getattr(config, "deployment", None), "compute_target", "") or ""
68         )
69         if target != "cloud_run":
70             return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
71
72         # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
73         selected.
74         from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
75
76         job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
77         region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
78         project = os.environ.get("GOOGLE_CLOUD_PROJECT")
79         client = run_client if run_client is not None else GoogleCloudRunJobClient()
80         return CloudRunCompute(
81             run_client=client,
82             job_name=job_name,
83             region=region,
84             project=project,
85             storage_config=_storage_dict(config),
86             policy=policy,
87             token=token,
88         )
89
90

```

All evidence gathered. Composing the section now.

26. user (2026-07-04T11:54:03.519Z)

Structured output provided successfully